



Programação Orientada a Objetos II

Profª. Angela Abreu Rosa de Sá, Dra.

Contato: angelaabreu@gmail.com

Horário das Aulas

Segunda-feira: 19:00hs – 20:40hs

→ *Lista de presença*

Lista de Presença

- Lista de presença em **TODAS** as aulas;
- As faltas **são lançadas** no Portal do Aluno;
- Abono de falta: **somente com atestado**;
- **CONFERIR** o total de faltas no Portal do Aluno.

Avaliações

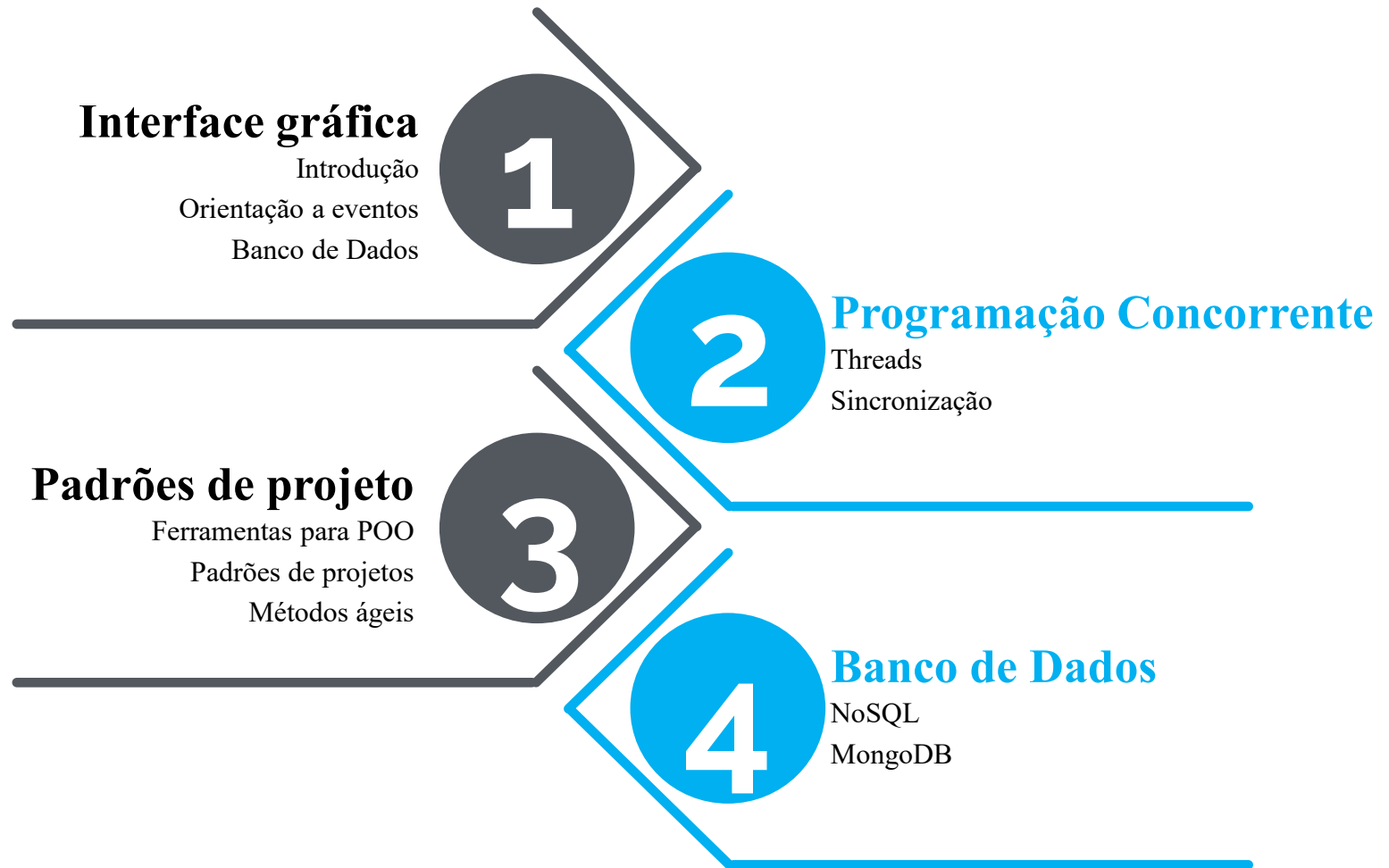
2 Avaliações Oficiais

- 2000 pontos **28 Setembro**
- 4000 pontos **23 Novembro**

Trabalhos: 2500 pontos

- 1000 pontos
- 1500 pontos

Ementa



Material Didático



Material Didático



Programação Orientada a Objetos II

O que é? Para que serve?



Programação Orientada a Objetos

Definição

**Orientação a Objetos é um
Paradigma de Programação.**



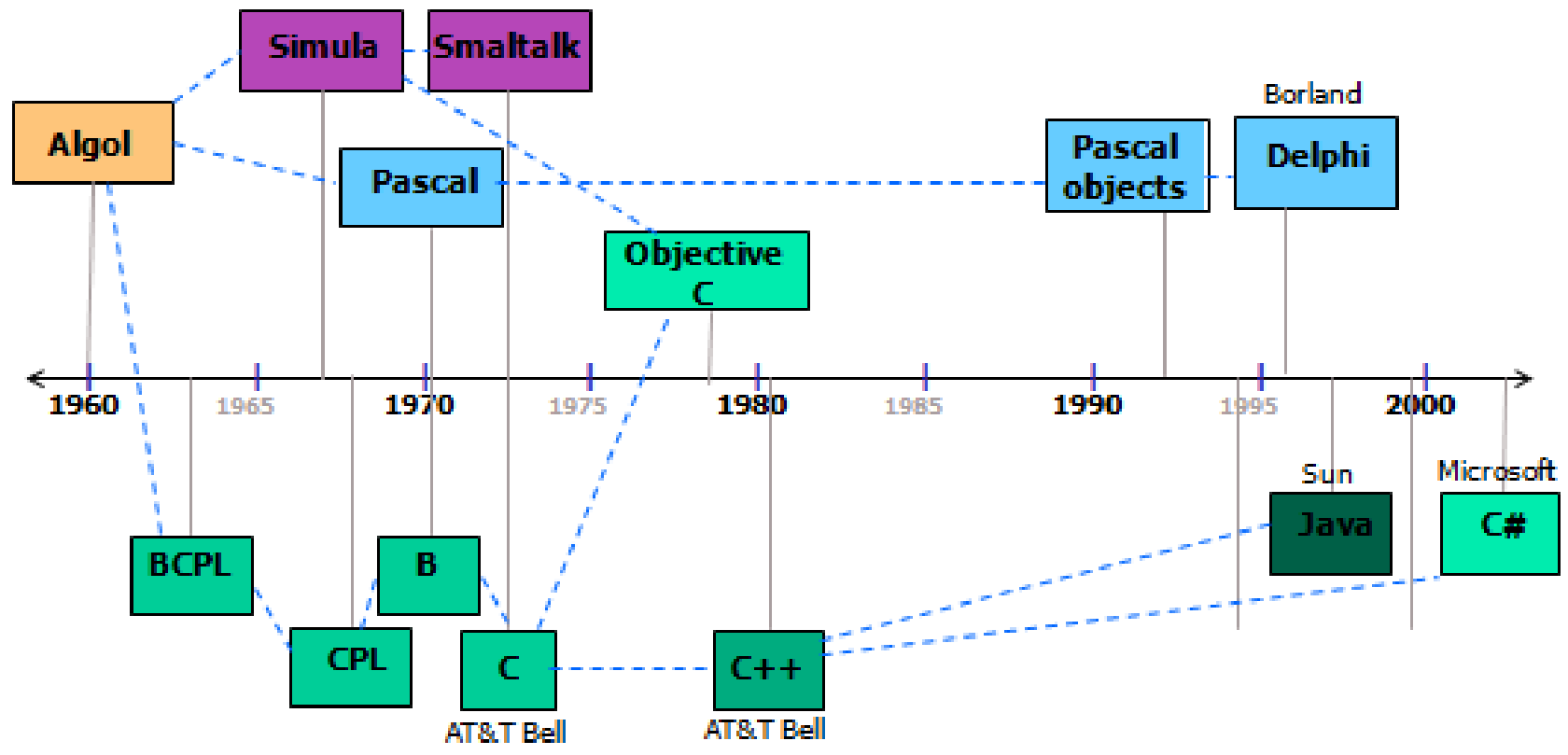
Paradigma?

Dicionário Aurélio: modelo, padrão

Paradigmas de Programação



Evolução das Linguagens de Programação



Vantagens da POO

- ✦ **Possuem vantagens sobre o paradigma estruturado:**
 - ✦ Reutilização de Software (Reusibilidade)
 - ✦ reaproveitamento de código
 - ✦ Manutenção de Software
 - ✦ facilidade de manutenção de sistemas
 - ✦ Tamanho de Código Gerado
 - ✦ menor código

Qual é a necessidade da POO?



Casa de
canhorro



Casa



Arranha-Ceús

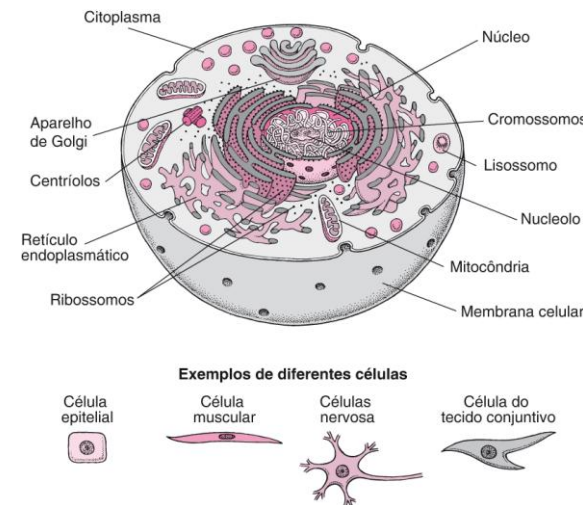
Aumento da complexidade

Origem

- O paradigma O.O. surgiu no **fim dos anos 60**.

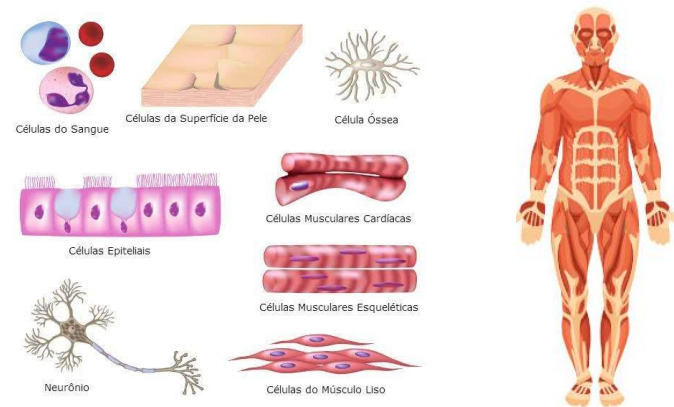
Alan Kay formulou a chamada **analogia biológica**.

“Como seria um sistema de software que funcionasse como um ser vivo?”



Origem

- Cada “célula” interagiria com outras células através do **envio de mensagens**
- Cada célula se comportaria como uma **unidade autônoma**.



- Kay pensou em como construir um sistema de software a partir de agentes autônomos que interagem entre si.

Fundamentos da O.O.

1. Qualquer **coisa** é um objeto.
2. Objetos realizam **tarefas** através da requisição de serviços a outros objetos.
3. Cada **objeto** pertence a uma determinada **classe**. Uma classe agrupa objetos similares.
4. A classe é um repositório para **comportamento** associado ao objeto.
5. Classes são organizadas em **hierarquias**.

Origem

Um sistema de software orientado a objetos consiste de **objetos em colaboração**



com o objetivo de **realizar as funcionalidades deste sistema**.



Cada **objeto** é responsável por **tarefas específicas**



É através da **cooperação entre objetos** que a computação do sistema se desenvolve.

Já aprendemos os conceitos da O.O na infância



O que é um objeto??????

O que é um objeto?



Possuem características comuns?

Objetos cachorros



BOB

Anda
Fala
Come
Dorme
PegaOsso



SCOOBY

Anda
Fala
Come
Dorme
PegaOsso



REX

Anda
Fala
Come
Dorme
PegaOsso

Classe e Objeto

Diferentes classes = Diferentes Objetos



*Características próprias => **ATRIBUTOS***

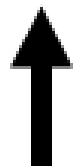
Classe e Objeto

Objetos e Classes

Palio	JWO-4567
--------------	-----------------

Parati	KLJ-0978
---------------	-----------------

Celta	JDK-6543
--------------	-----------------


OBJETOS

(Instâncias da classe Automóvel)

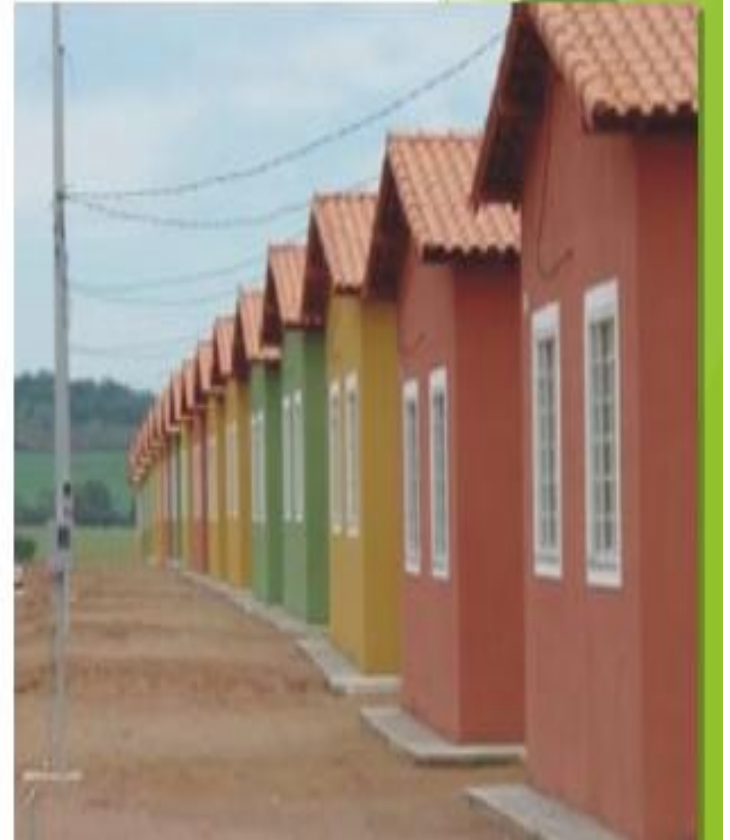
Automóvel
Marca
Placa


CLASSE

Classe e Objeto



Objeto



Objetos

CLASSE

Life: 5

Pontuação: 2

Tempo: 28



Normal - UserForm1 (UserForm)

UserForm1

Toolbox

Controls



 Central *Subject*

Nova Solicitação:

Nome:

Rua:

Nº:

Bairro:

Complemento:

Destino:

Limpar

Solicitar>>>

 Taxi *Observer* - #0001

Cliente:

Nome:

Endereço:

Destino:

 Taxi *Observer* - #0002

Cliente:

Nome:

Endereço:

Destino:

 Taxi *Observer* - #0003

Cliente:

Nome:

Endereço:

Destino:

Conceitos Fundamentais

- Classe / Objeto
- Construtor
- Atributos
- Métodos
- Encapsulamento
- Generalização/Especialização
- Herança
- Polimorfismo e sobrecarga

Conceitos Fundamentais

- Uma **classe** é um molde para objetos.
- Um objeto é uma instância de uma classe.
- Uma classe é uma *abstração* das características *relevantes* de um grupo de coisas do mundo real.

Pilares da O.O.



Abstração

- Uma abstração é uma representação das características e do comportamento relevantes de um conceito do mundo real para um determinado problema.

Mensagem

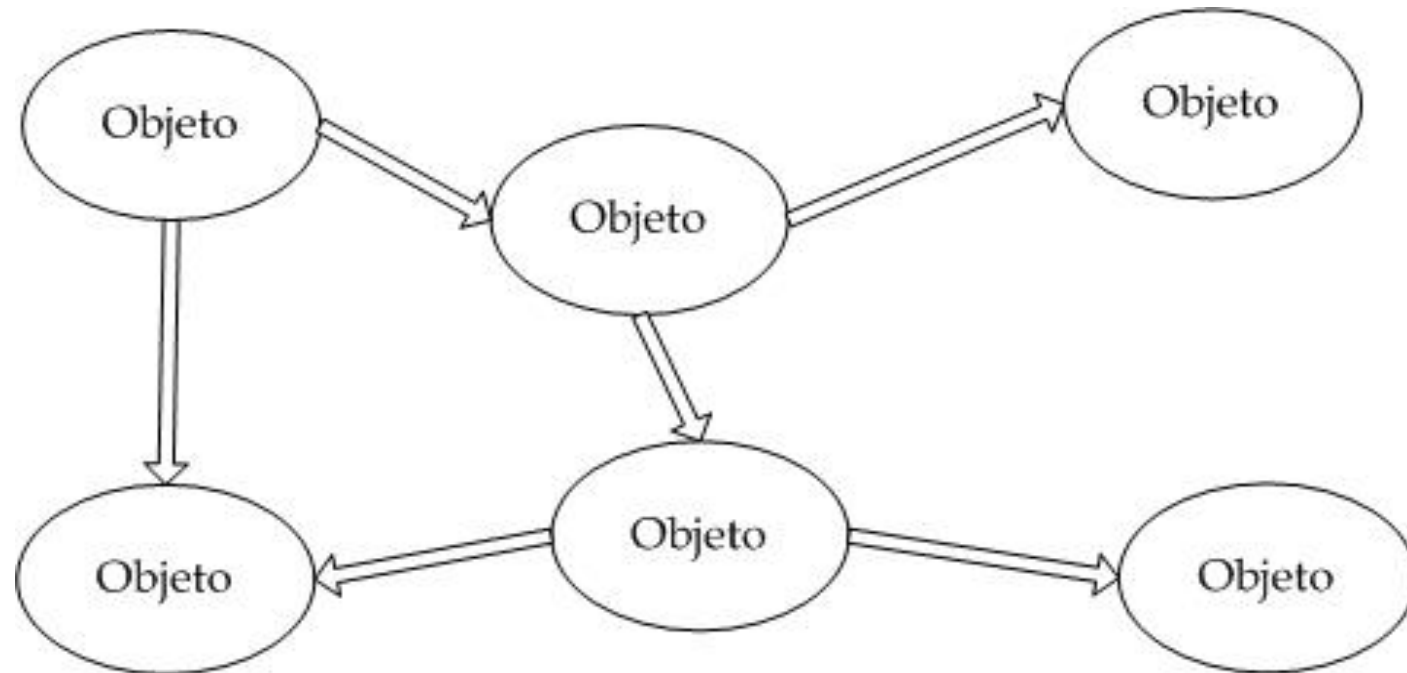
- Para que um objeto realize alguma tarefa, deve haver um **estímulo** enviado a este objeto.



Mensagem

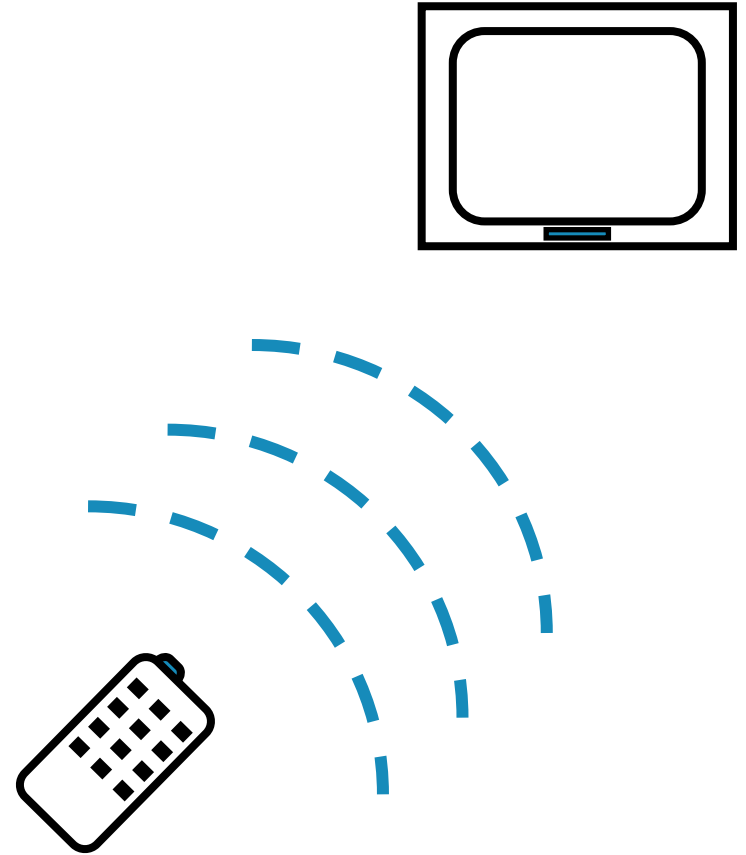
- Uma mensagem é uma requisição enviada de **um objeto a outro** para que este **último** realize alguma operação.

Mensagem



Encapsulamento

- Objetos devem “**esconder**” a sua complexidade.
 - Legibilidade
 - Clareza
 - Reuso



Deixar **PÚBLICO** somente o que é necessário para enviar mensagens para o objeto.

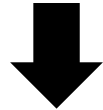
Encapsulamento

VISIBILIDADE da classe

Public

Private

Modificadores de Acesso



Quem pode “ver” e acessar o conteúdo de cada atributo e método?
Somente a classe ou o programa principal?

Encapsulamento

Tipos de Visibilidade

■ Público **Public**

Os Atributos e Métodos que PÚBLICOS, são **visíveis** para o **programa** que está utilizando a classe

■ Privado **Private**

Os Atributos e Métodos que PRIVATE, são **visíveis** para apenas **DENTRO da classe**

Encapsulamento

Ocultação de detalhes internos



Tornar a classe uma **CÁPSULA**, isto é, uma Caixa Preta



Deixar visível somente o **NECESSÁRIO** para a manipulação da classe.

Encapsulamento

```
public class Calculadora {  
  
    public int num1;  
    public int num2;  
  
    Calculadora()  
    {  
        num1 = 0;  
        num2 = 0;  
    }  
  
    Calculadora(int n1,int n2)..  
  
    Calculadora (int n)..  
  
    public int Somar()  
    {  
        return num1 + num2;  
    }  
  
    public int Somar(int n1, int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
  
        return num1 + num2;  
    }  
}
```



```
public class Principal01 {  
  
    public static void main(String[] args) {  
  
        Calculadora obj1 = new Calculadora();  
  
        obj1.num1 = 2;  
        obj1.num2 = 4;  
  
        int somatorio = obj1.Somar();  
  
    }  
}
```

Encapsulamento

```
public class Calculadora {  
  
    private int num1;  
    private int num2;  
  
    Calculadora()  
    {  
        num1 = 0;  
        num2 = 0;  
    }  
  
    Calculadora(int n1, int n2) {  
  
    }  
  
    Calculadora (int n) {  
  
    }  
  
    public int Somar()  
    {  
        return num1 + num2;  
    }  
  
    public int Somar(int n1, int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
  
        return num1 + num2;  
    }  
}
```



```
public class Principal01 {  
  
    public static void main(String[] args) {  
  
        Calculadora obj1 = new Calculadora();  
  
        obj1.num1 = 2;  
        obj1.num2 = 4;  
  
        int somatorio = obj1.Somar();  
    }  
}
```

Problems × @ Javadoc Declaration

Errors, 1 warning, 0 others

Description

Resource

Errors (2 items)

The field Calculadora.num1 is not visible

Principal01.java

The field Calculadora.num2 is not visible

Principal01.java

Warnings (1 item)

Build path specifies execution environment JavaSE- Ex01

Atributos **não VISÍVEIS** para
outras classes. **Não podem ser
acessados diretamente!**

Encapsulamento

```
public class Calculadora {
```

```
    private int num1;  
    private int num2;
```

```
    Calculadora()  
    {  
        num1 = 0;  
        num2 = 0;  
    }
```

```
    Calculadora(int n1, int n2) {
```

```
    Calculadora (int n) {
```

```
    public int Somar()  
    {  
        return num1 + num2;  
    }
```

```
    public int Somar(int n1, int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
  
        return num1 + num2;  
    }
```



```
public class Principal01 {
```

```
    public static void main(String[] args) {
```

```
        Calculadora obj1 = new Calculadora(2,4);
```

```
        int somatorio = obj1.Somar();
```

```
        //outra opcao
```

```
        Calculadora obj2 = new Calculadora();
```

```
        int somatorio2 = obj2.Somar(2, 4);
```

1

Atributos **PRIVATE** só serão
acessados e alterados do
métodos **PUBLIC**

Encapsulamento

```
public class Calculadora {  
  
    private int num1;  
    private int num2;  
  
    Calculadora(int n1, int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
    }  
  
    private int Somar()  
    {  
        return num1 + num2;  
    }  
  
    public double Media()  
    {  
        int soma = Somar();  
  
        double media = soma/2;  
  
        return soma;  
    }  
}
```

Os atributos são **PRIVADOS**...

E se precisar alterar o conteúdo deles depois da criação do objeto?

E se precisar saber qual é o conteúdo destes atributos?

```
public class Calculadora {
```

```
    private int num1;  
    private int num2;
```

```
    Calculadora(int n1,int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
    }
```

```
    private int Somar()  
    {  
        return num1 + num2;  
    }
```

```
    public double Media()  
    {  
        int soma = Somar();  
  
        double media = soma/2;  
  
        return soma;  
    }
```

```
    public void AlterarNumeros(int n1, int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
        return;  
    }
```

```
    public int RetornaNumero1()  
    {  
        return num1;  
    }
```

```
    public int RetornaNumero2()  
    {  
        return num2;  
    }
```

Métodos **PÚBLICOS** do tipo “**GET**
e SET” para acessar e retornar o
conteúdo de atributos que são
PRIVATE.

Herança

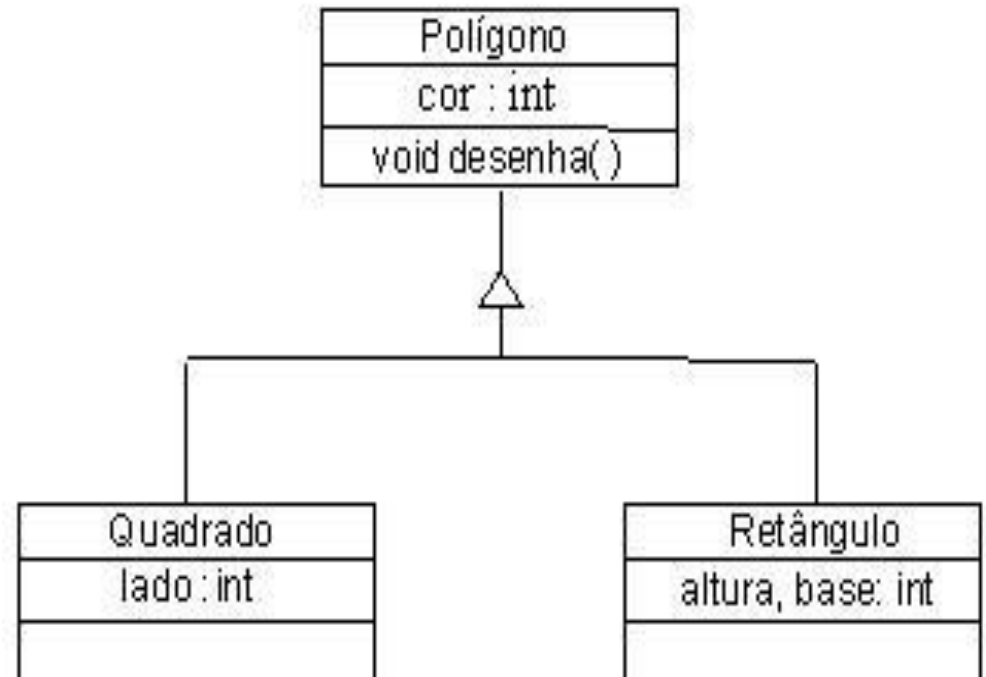
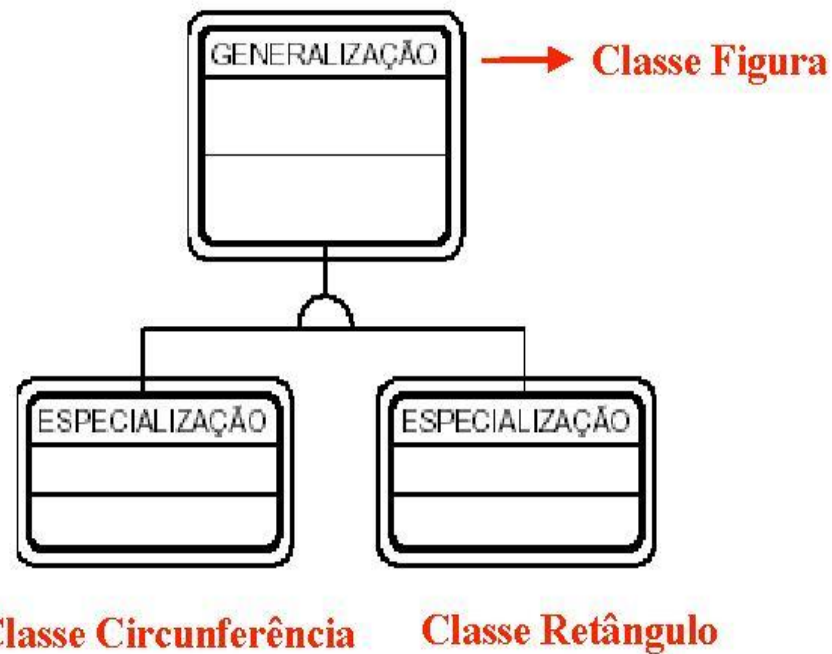
“ É um mecanismo para derivar novas classes a partir de classes existentes”



Superclasse ou classe pai

Subclasse ou classe filha

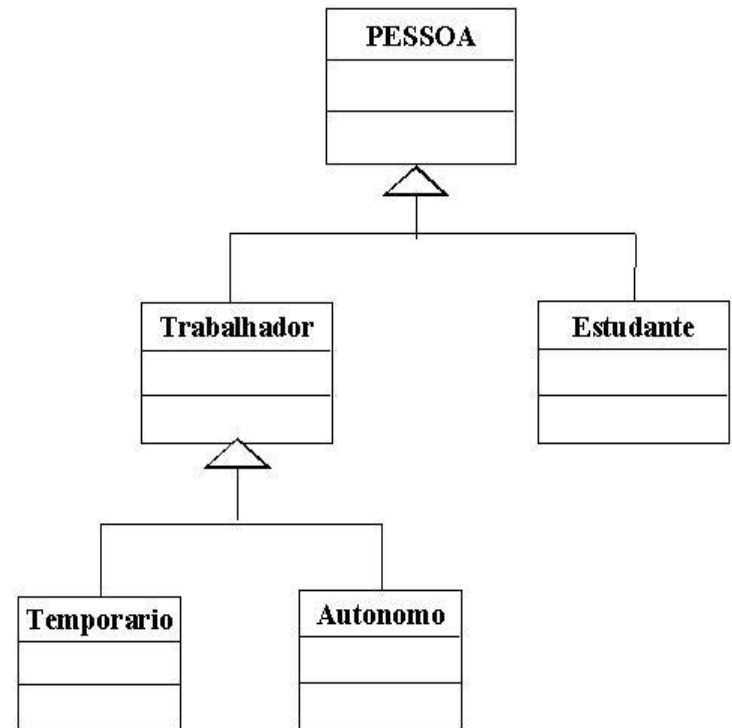
Herança



Herança

- Redução no custo de manutenção;
- Aumento na reutilização de código ;
- Modularização;

Vantagens

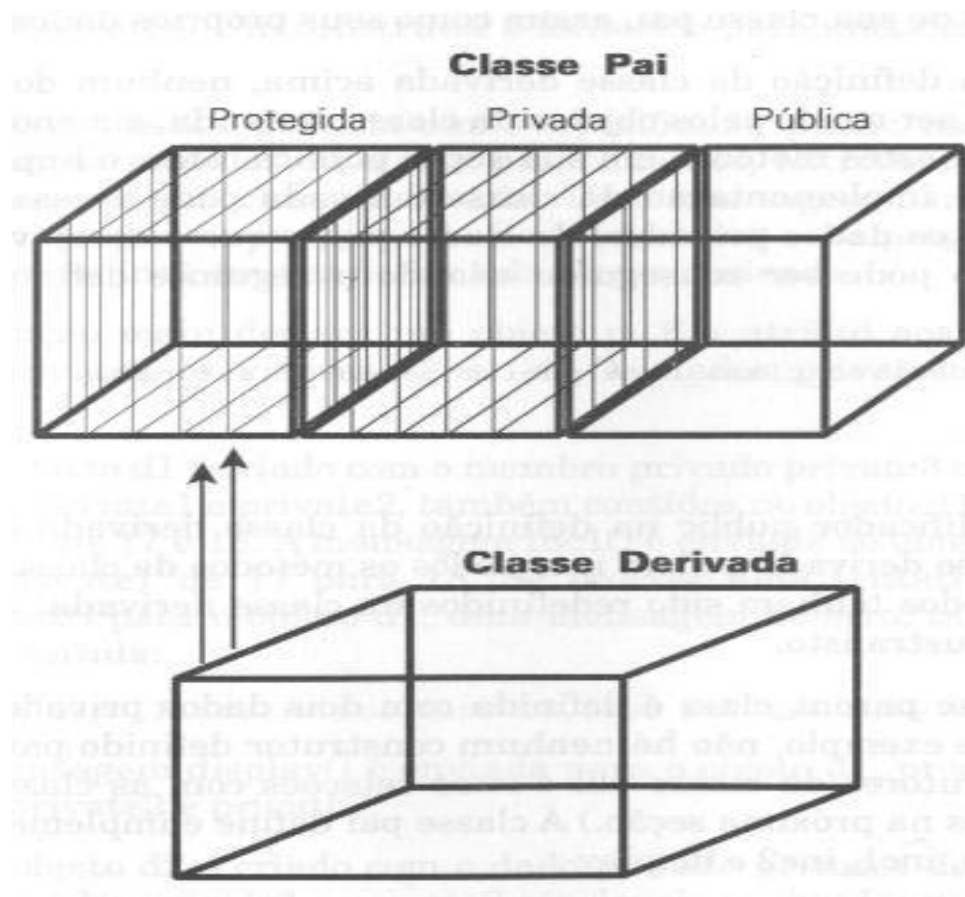


Herança

- Permite modelar uma hierarquia entre classes: **classes filhas herdam** propriedades (atributos e métodos) da **classe pai**;
- Identifica-se a possibilidade de herança através da seguinte expressão típica: **é um tipo de**.

Herança

Tipos de acesso



Herança

Tipo de acesso	Descrição
Public	Estes atributos e métodos são sempre acessíveis em todos os métodos de todas as classes. Este é o nível menos rígido de encapsulamento, que equivale a não encapsular.
Private	Esses atributos e métodos são acessíveis somente nos métodos da própria classe. Este é o nível mais rígido de encapsulamento.
<u>Protected</u>	Esses atributos e métodos são acessíveis nos métodos da própria classe e suas subclasses.

Herança

Funcionário

Nome, cargo, data
de admissão

```
public class Funcionario {  
  
    String nome;  
    String cpf;  
    double salario;  
  
}
```

Gerente

Nome, cargo, data
de admissão, Quantidade
de funcionários que
gerencia, senha de acesso.

```
public class Gerente extends Funcionario {  
  
    int senha;  
    int numeroDeFuncionariosGerenciados;  
  
}
```

```
public class Funcionario {  
  
    protected String nome;  
    protected String cpf;  
    protected double salario;  
  
    public Funcionario(String n, String numcpf, double sal)  
    { //inicializar os atributos  
        nome = n;  
        cpf = numcpf;  
        salario = sal;  
    }  
}
```

```
public class Gerente extends Funcionario {  
  
    private int senha;  
    private int numeroDeFuncionariosGerenciados;  
  
    public Gerente (String n, String numcpf, double sal, int s, int numgerenciado)  
    {  
        super(n,numcpf,sal); //chamando o construtor da super classe (classe pai)  
        senha = s;  
        numeroDeFuncionariosGerenciados = numgerenciado;  
    }  
}
```

```
public class Pessoa {  
    //atributos  
    protected String nome;  
  
    //construtor  
    public Pessoa(String nome)  
    {  
        this.nome = nome;  
    }  
    //métodos  
    public String getNome() {  
        return nome;  
    }  
}
```

```
public class PessoaFisica extends Pessoa {  
    //atributos  
    private long cpf;  
  
    //construtor  
    public PessoaFisica(String nome, long cpf) {  
  
        super(nome); //chamando o construtor da classe pai  
        this.cpf = cpf;  
    }  
  
    //métodos  
    public String getNome() {  
        return "Pessoa Fisica: " + super.getNome() + " - CPF: " + cpf;  
    }  
}
```

Polimorfismo

poli = muitas,
morphos = formas

POLIMORFISMO

MUITAS FORMAS

Polimorfismo

- Resumindo: “Vamos nos adaptar”.
- Permite a um método ter **várias implementações** as quais são selecionadas com base na **quantidade de parâmetros** e **seus tipos** que é passado para a invocação do método.

Janela ()

Janela (1 x 2 , 2)

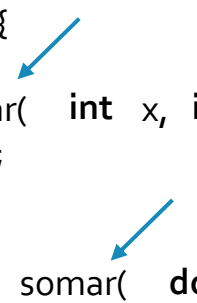
Janela (1 x 2 , 2, Azul)

Ou sobrecarga?

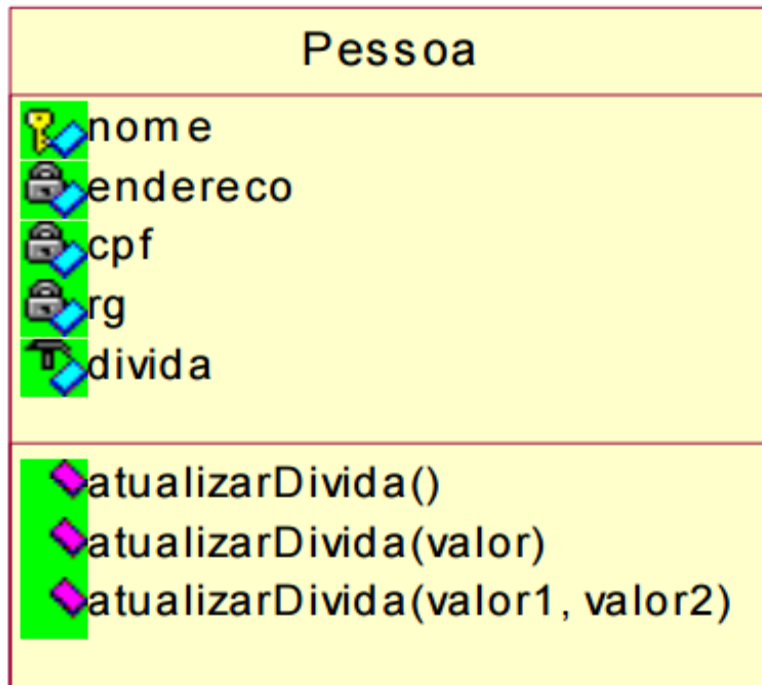
Polimorfismo

- Permite a existência de vários métodos de mesmo **nome**, porém com **assinaturas** levemente diferentes (número, tipo e qtd de parâmetros).

```
public class Soma {  
    public int somar( int x, int y){  
        return x+y;  
    }  
  
    public double somar( double x, double y) {  
        return x+y;  
    }  
}
```



Polimorfismo



Três métodos com o mesmo nome, porém, são diferenciados devido a quantidade de parâmetros passados

Polimorfismo

- Observe que são declarados 3 construtores e todos eles têm o mesmo nome da classe. Entretanto, cada um deles recebe um número diferente de parâmetros.

```
public class Pessoa {  
    private String rg;  
    private String nome;  
  
    public Pessoa() {  
    }  
    public Pessoa(String rg, String nome) {  
        this.rg = rg;  
        this.nome = nome;  
    }  
    public Pessoa(String nome) {  
        this.nome = nome;  
    }  
}
```

```
public class Calculadora {
```

```
    private int num1;  
    private int num2;
```

```
    Calculadora(int n1,int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
    }
```

```
    private int Somar()  
    {  
        return num1 + num2;  
    }
```

```
    public double Media()  
    {  
        int soma = Somar();  
  
        double media = soma/2;  
  
        return soma;  
    }
```

```
    public void AlterarNumeros(int n1, int n2)  
    {  
        num1 = n1;  
        num2 = n2;  
        return;  
    }
```

```
    public int RetornaNumero1()  
    {  
        return num1;  
    }
```

```
    public int RetornaNumero2()  
    {  
        return num2;  
    }
```

Métodos **PÚBLICOS** do tipo “**GET**
e SET” para acessar e retornar o
conteúdo de atributos que são
PRIVATE OU PROTECTED.

```
public class Funcionario {
```

```
    protected String nome;
```

```
    1  Undo Typing          Ctrl+Z
```

```
    1  Revert File
```

```
    1  Save                Ctrl+S
```

```
    Open Declaration      F3
```

```
    Open Type Hierarchy   F4
```

```
    Open Call Hierarchy   Ctrl+Alt+H
```

```
    Show in Breadcrumb    Alt+Shift+B
```

```
    Quick Outline         Ctrl+O
```

```
    Quick Type Hierarchy  Ctrl+T
```

```
    Open With             >
```

```
    Show In               Alt+Shift+W >
```

```
✂ Cut                    Ctrl+X
```

```
📄 Copy                  Ctrl+C
```

```
📄 Copy Qualified Name
```

```
📄 Paste                 Ctrl+V
```

```
📄 Raw Paste
```

```
Quick Fix               Ctrl+1
```

```
Source                  Alt+Shift+S >
```

```
Refactor                Alt+Shift+T >
```

```
Local History           >
```

```
References              >
```

```
Declarations            >
```

```
📊 Coverage As          >
```

```
🏃 Run As               >
```

```
🔍 Debug As             >
```

```
Team                    >
```

```
Compare With            >
```

```
Replace With            >
```

```
Preferences...
```

```
Toggle Comment          Ctrl+7
```

```
Remove Block Comment    Ctrl+Shift+\
```

```
Generate Element Comment Alt+Shift+J
```

```
Correct Indentation      Ctrl+I
```

```
Format                   Ctrl+Shift+F
```

```
Format Element
```

```
Add Import              Ctrl+Shift+M
```

```
Organize Imports         Ctrl+Shift+O
```

```
Sort Members...
```

```
Clean Up...
```

```
Override/Implement Methods...
```

```
Generate Getters and Setters...
```

```
Generate Delegate Methods...
```

```
Generate hashCode() and equals()...
```

```
Generate toString()...
```

```
Generate Constructor using Fields...
```

```
Generate Constructors from Superclass...
```

```
Externalize Strings...
```

jdk-18.0.1.1\bin\javaw.exe (24 de ago. de 2022 11:50:33 – 11:50:35) [pid: 19364]

Writable

Smart Insert

16 : 5 : 270

Generate Getters and Setters

Select getters and setters to create:

- > ☒ cpf
- > ☒ nome
- > ☒ salario

Select All

Deselect All

Select Getters

Select Setters

☐ Allow setters for final fields (remove 'final' modifier from fields if necessary)

Insertion point:

After 'Funcionario(String, String, double)'

Sort by:

Fields in getter/setter pairs

Access modifier

☒ public ☐ protected ☐ package ☐ private

☐ final ☐ synchronized

☐ Generate method comments

The format of the getters/setters may be configured on the [Code Templates](#) preference page.

6 of 6 selected.



Generate

Cancel

Outline ×

Funcion

nome

cpf : :

salari

Funci

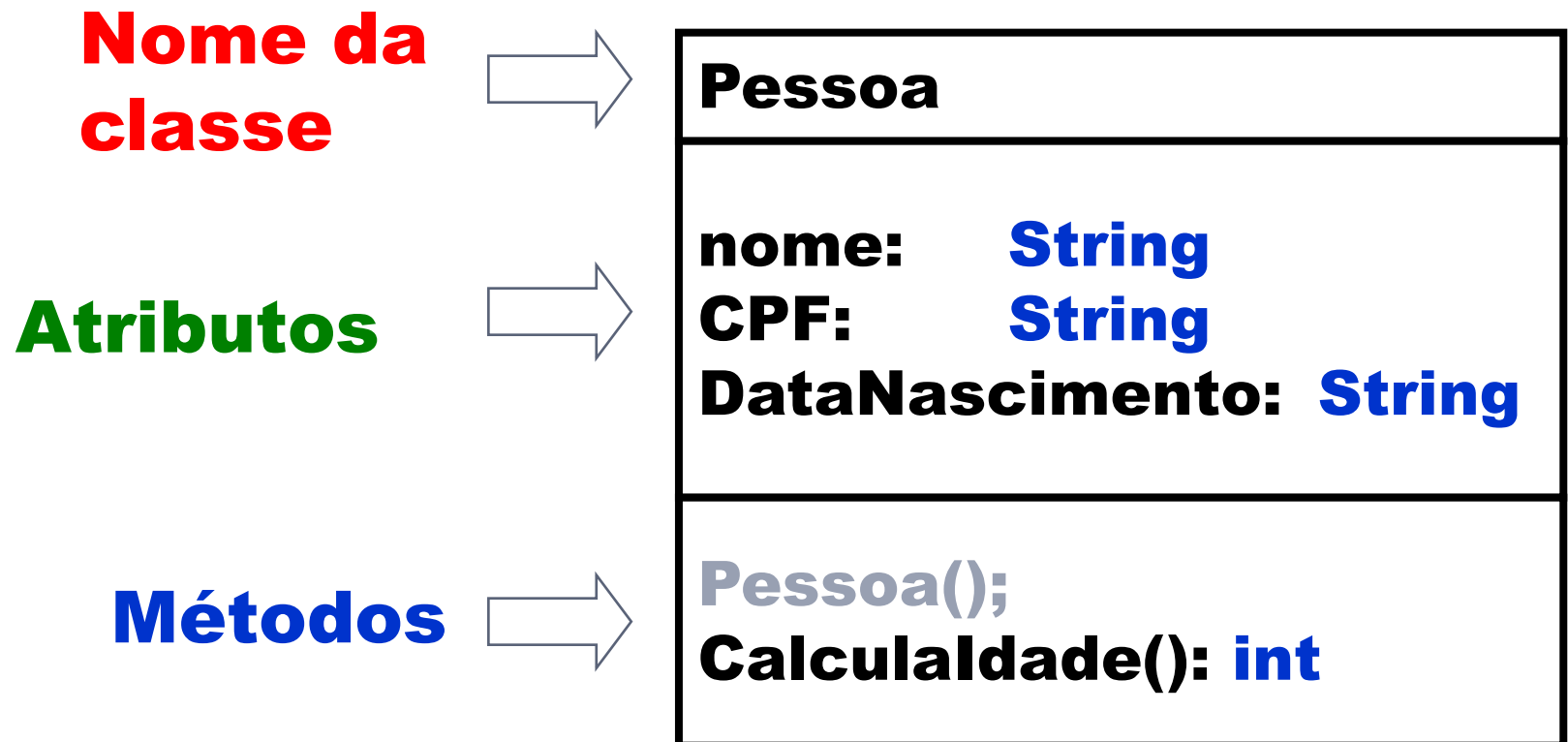
- 11:50:35) [pid: 19364]

```
public class Funcionario {  
  
    protected String nome;  
    protected String cpf;  
    protected double salario;  
  
    public Funcionario(String n, String numcpf, double sal)  
    { //inicializar os atributos  
        nome = n;  
        cpf = numcpf;  
        salario = sal;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public String getCpf() {  
        return cpf;  
    }  
  
    public void setCpf(String cpf) {  
        this.cpf = cpf;  
    }  
  
    public double getSalario() {  
        return salario;  
    }  
  
    public void setSalario(double salario) {  
        this.salario = salario;  
    }  
}
```

this

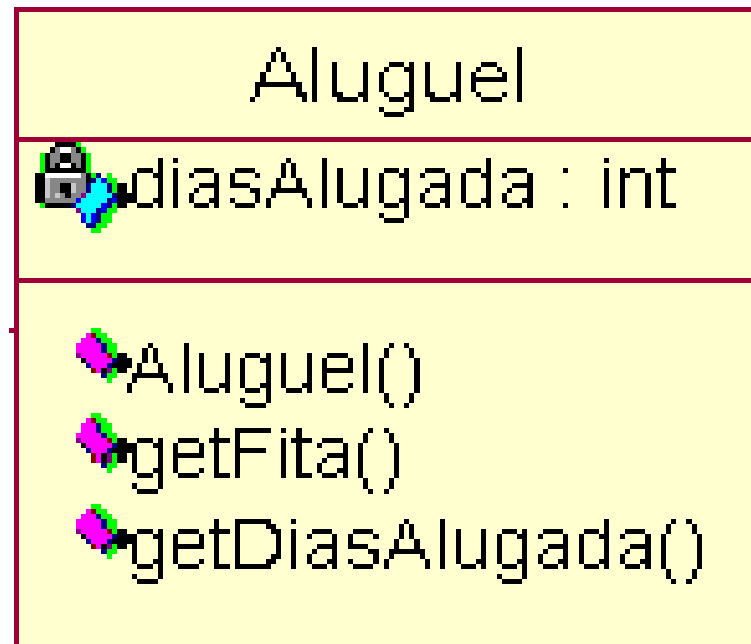
Linguagem de Modelagem Universal (UML)

UML (Unified Modeling Language)

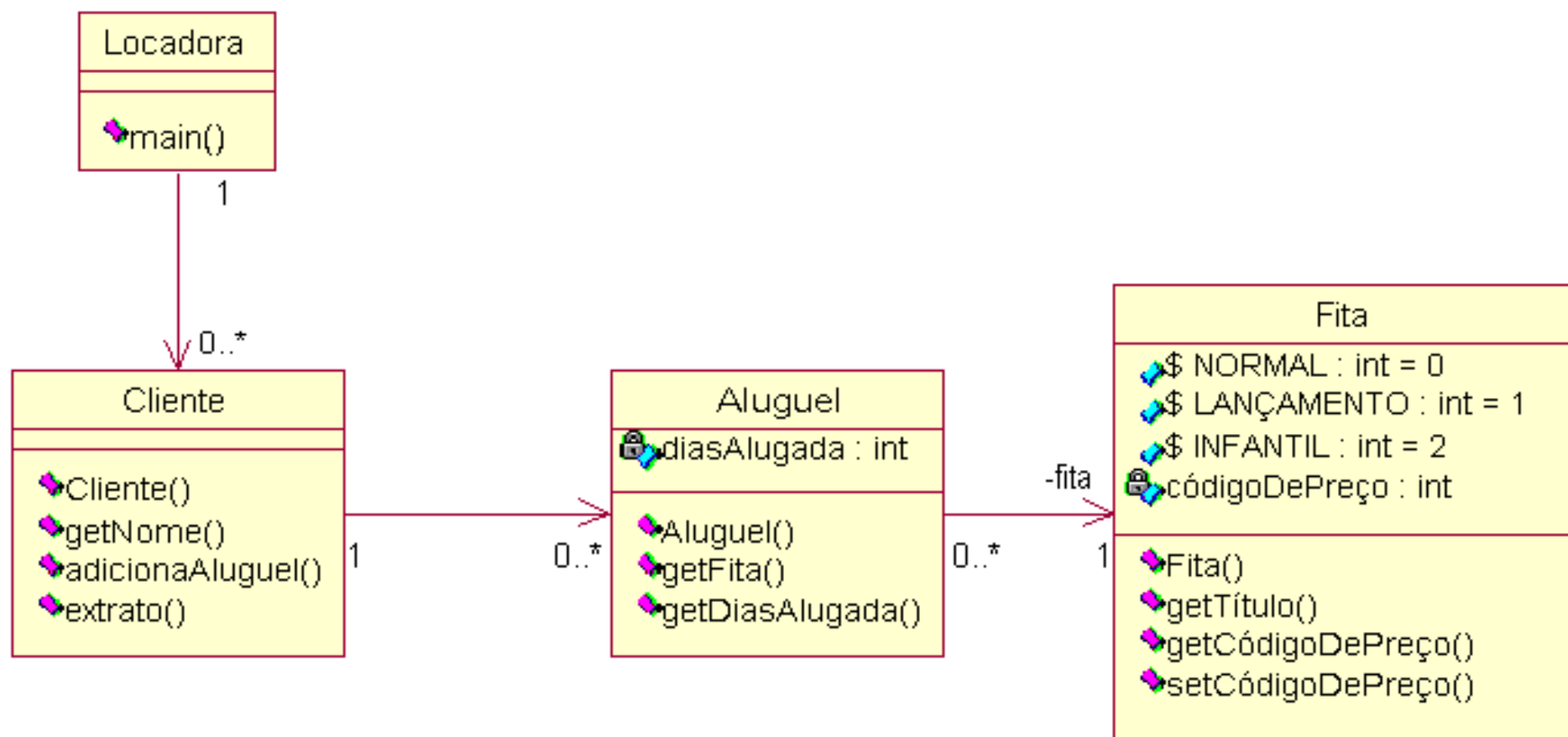


Modelagem O.O. em UML

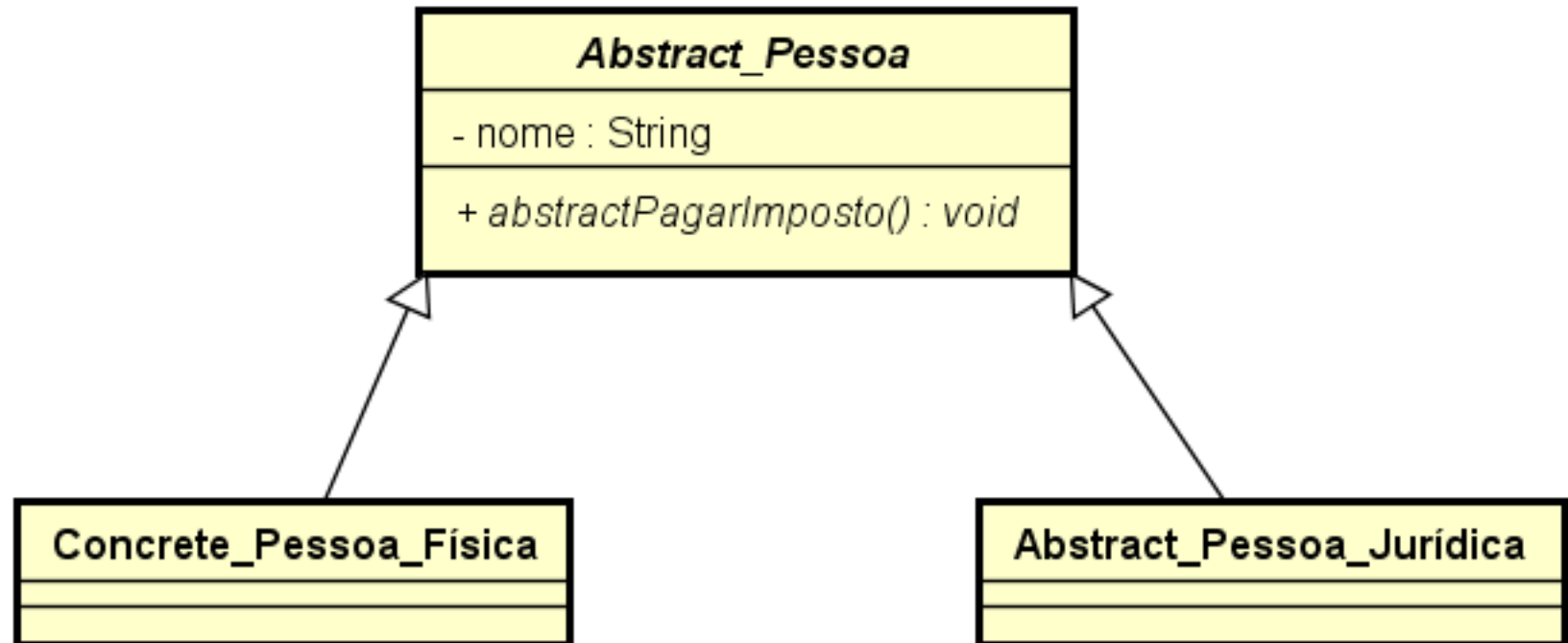
Linguagem UML (Unified Modeling Language)

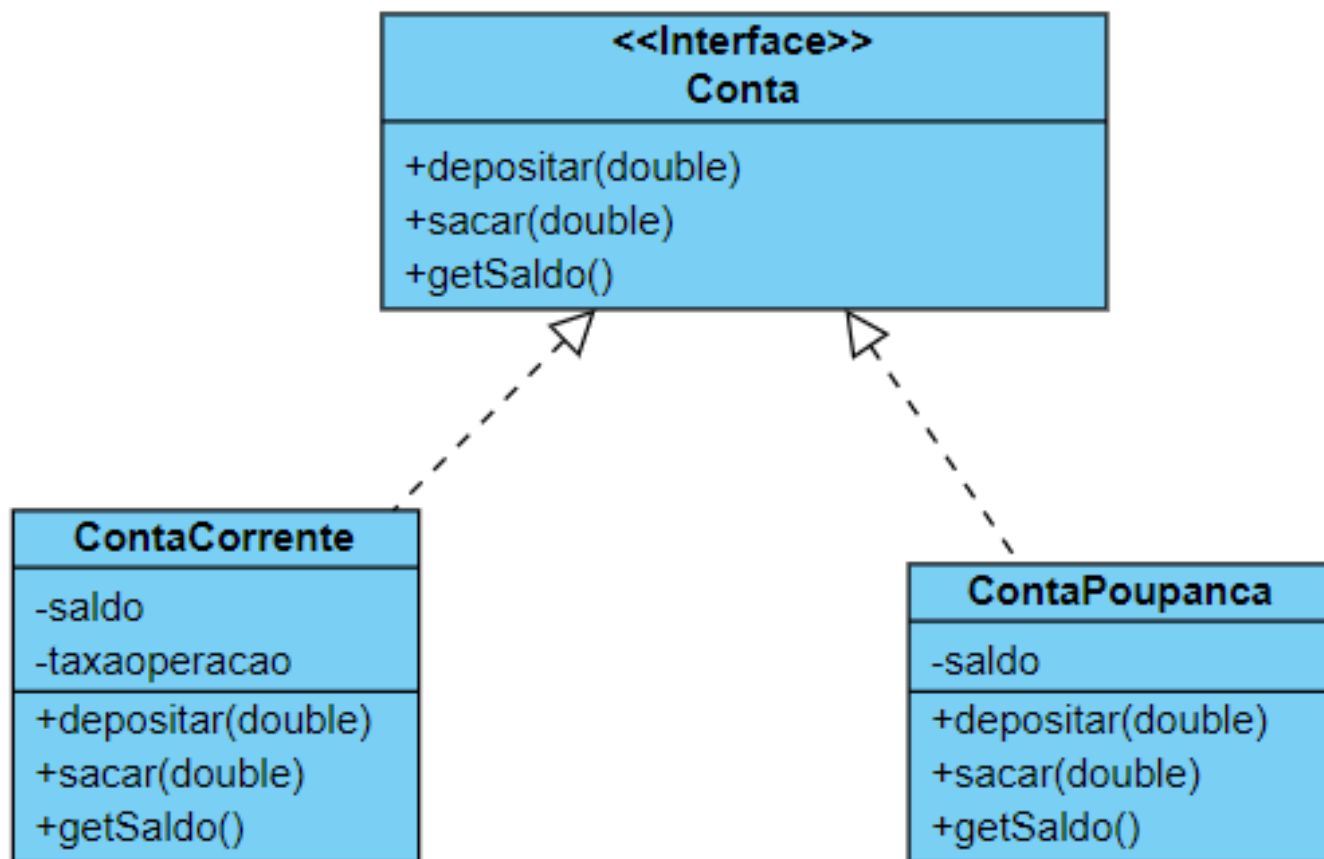


Modelagem O.O. em UML



Classes Abstratas





Interfaces

Diferença básica entre classes abstratas e interfaces :

Classe Abstrata

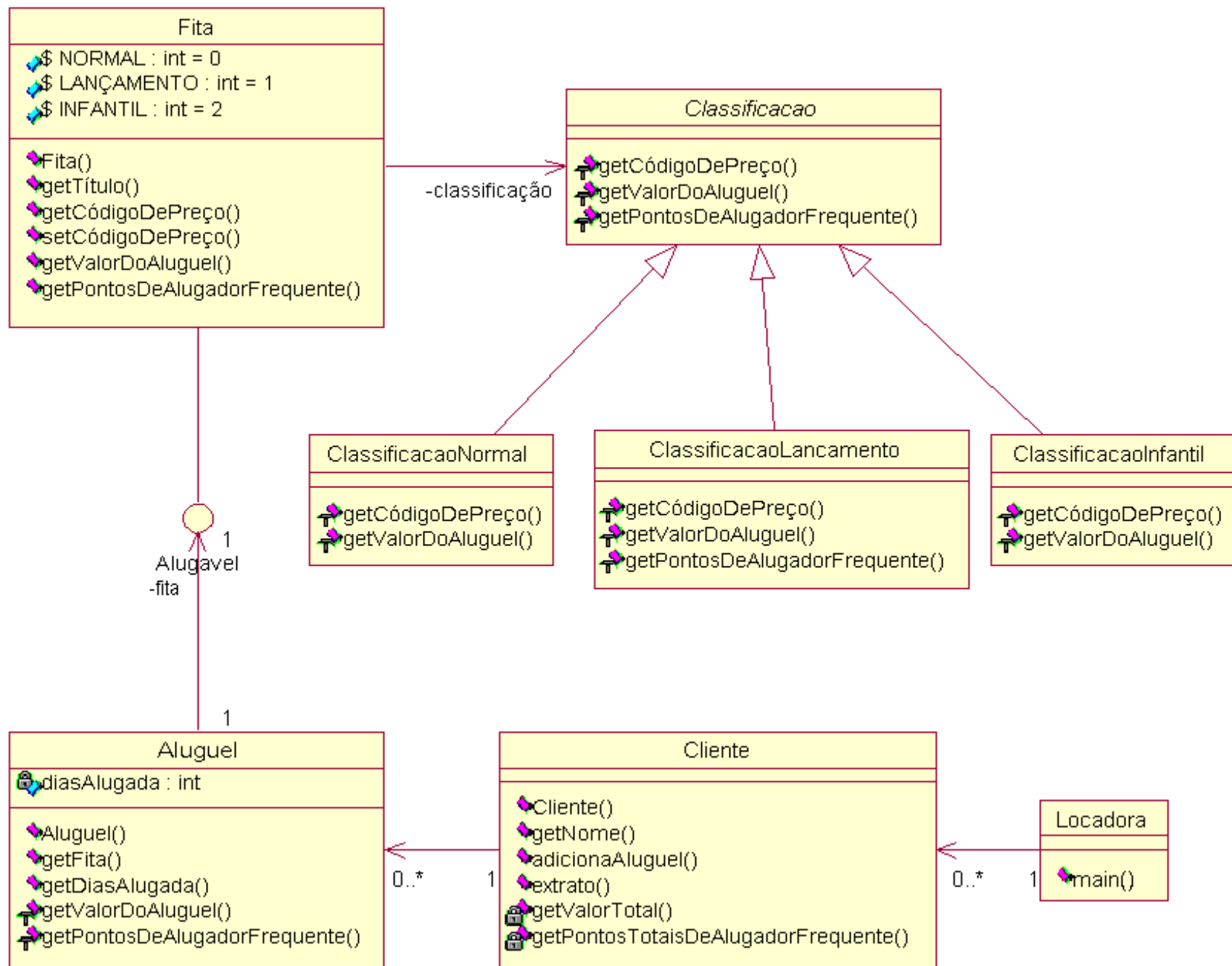


qualquer classe pode implementar várias interfaces simultaneamente.

Interface



classe herdeira somente **pode herdar de uma única classe (independentemente de ser abstrata ou não),**



```

import java.util.Scanner;

public class Principal01 {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);

        String nome;
        int nota1, nota2, nota3;

        System.out.println("***** Dados do Aluno ***** ");
        System.out.print("Digite o nome do aluno: ");
        nome = scan.next();
        System.out.print("Digite a nota 1: ");
        nota1 = scan.nextInt();
        System.out.print("Digite a nota 2: ");
        nota2 = scan.nextInt();
        System.out.print("Digite a nota 3: ");
        nota3 = scan.nextInt();

        Aluno objetoAluno; //declaração do objeto
        objetoAluno = new Aluno(nome, nota1,nota2,nota3);

        System.out.println("***** Boletim final ***** ");
        System.out.println("Aluno " + objetoAluno.RetornaNome() + " está " + objetoAluno.ResultadoFinal());

    }
}

```

<terminated> Principal01 [Java Application] C:\Program Files\

***** Dados do Aluno *****

Digite o nome do aluno: XXXX

Digite a nota 1: 8

Digite a nota 2: 8

Digite a nota 3: 8

***** Boletim final *****

Aluno XXXX está Aprovado

Ementa



Material Didático



Programação Orientada a Objetos II

Sumário

Unidade 1 Programação orientada a eventos com interfaces gráficas e banco de dados relacional	7
Seção 1.1 - Desenvolvimento de interfaces gráficas na linguagem Java	9
Seção 1.2 - Programação em Java usando orientação a eventos	27
Seção 1.3 - Programação em Java usando banco de dados relacional	47
Unidade 2 Programação concorrente orientada a objetos	67
Seção 2.1 - Programação em Java usando threads	69
Seção 2.2 - Definição e tratamento de exceções para sistemas com threads	89
Seção 2.3 - Programação em Java utilizando elementos para sincronização em Java	107
Unidade 3 Padrões de projeto, ferramentas e métodos ágeis	127
Seção 3.1 - Ferramentas para programação em linguagens orientadas a objetos	129
Seção 3.2 - Padrões de projetos em orientação a objetos	149
Seção 3.3 - Métodos ágeis em orientação a objetos	167
Unidade 4 Novas tecnologias para programação em banco de dados	183
Seção 4.1 - Banco de dados NoSQL	185
Seção 4.2 - Introdução ao desenvolvimento em Java usando MongoDB	200
Seção 4.3 - Desenvolvimento em Java usando MongoDB	216

Interface Gráfica

Console setup

- 1) Configure Network Interfaces
- 2) Configure Link Aggregation
- 3) Configure VLAN Interface
- 4) Configure Default Route
- 5) Configure Static Routes
- 6) Configure DNS
- 7) Reset Root Password
- 8) Reset Configuration to Defaults
- 9) Shell
- 10) Reboot
- 11) Shut Down
- 12) Toggle automatic support alerts to iXsystems

Sistema de Gerência de Taxis

Observers

Adicionar

Central *Subject*

Nova Solicitação:

Nome:

Rua:

Nº:

0

Bairro:

Complemento:

Destino:

Limpar

Solicitar>>>

Taxi *Observer* - #0001

Cliente:

Nome:

Endereço:

Destino:

Taxi *Observer* - #0002

Cliente:

Nome:

Endereço:

Destino:

Taxi *Observer* - #0003

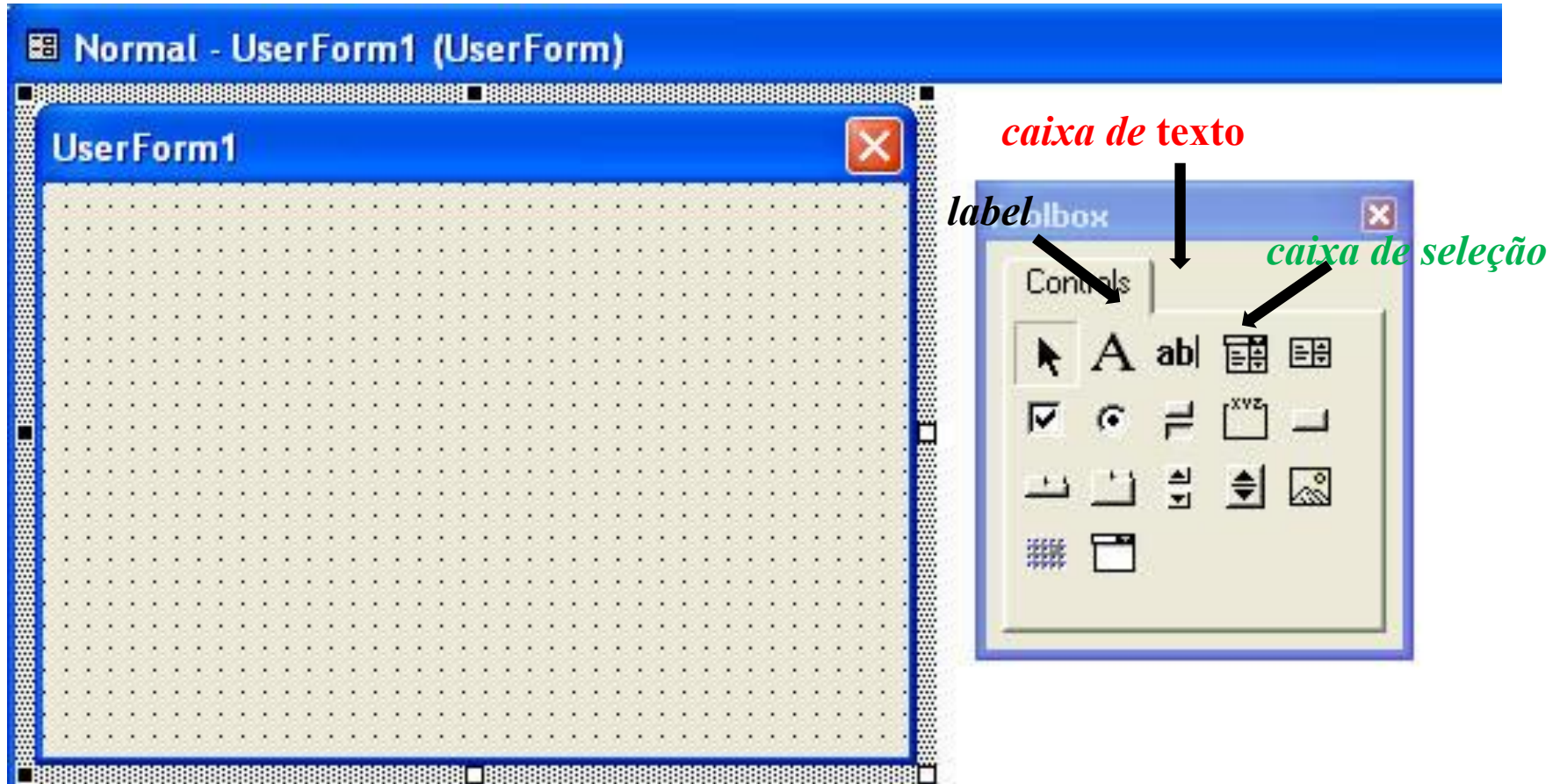
Cliente:

Nome:

Endereço:

Destino:

Interface Gráfica



What You See I What You Get **WYSIWYG**

Interface Gráfica

GUI - Graphical User Interface

Boas práticas

Posicionamento dos componentes

Tamanho da Janela

Cores

User friendly

UX User eXperience

UI User Interface

Interface Gráfica

Opção 1

Java Development Kit (JDK)

Swing *(biblioteca incorporada no JDK)*

Integrated Development Environment (IDE) Eclipse

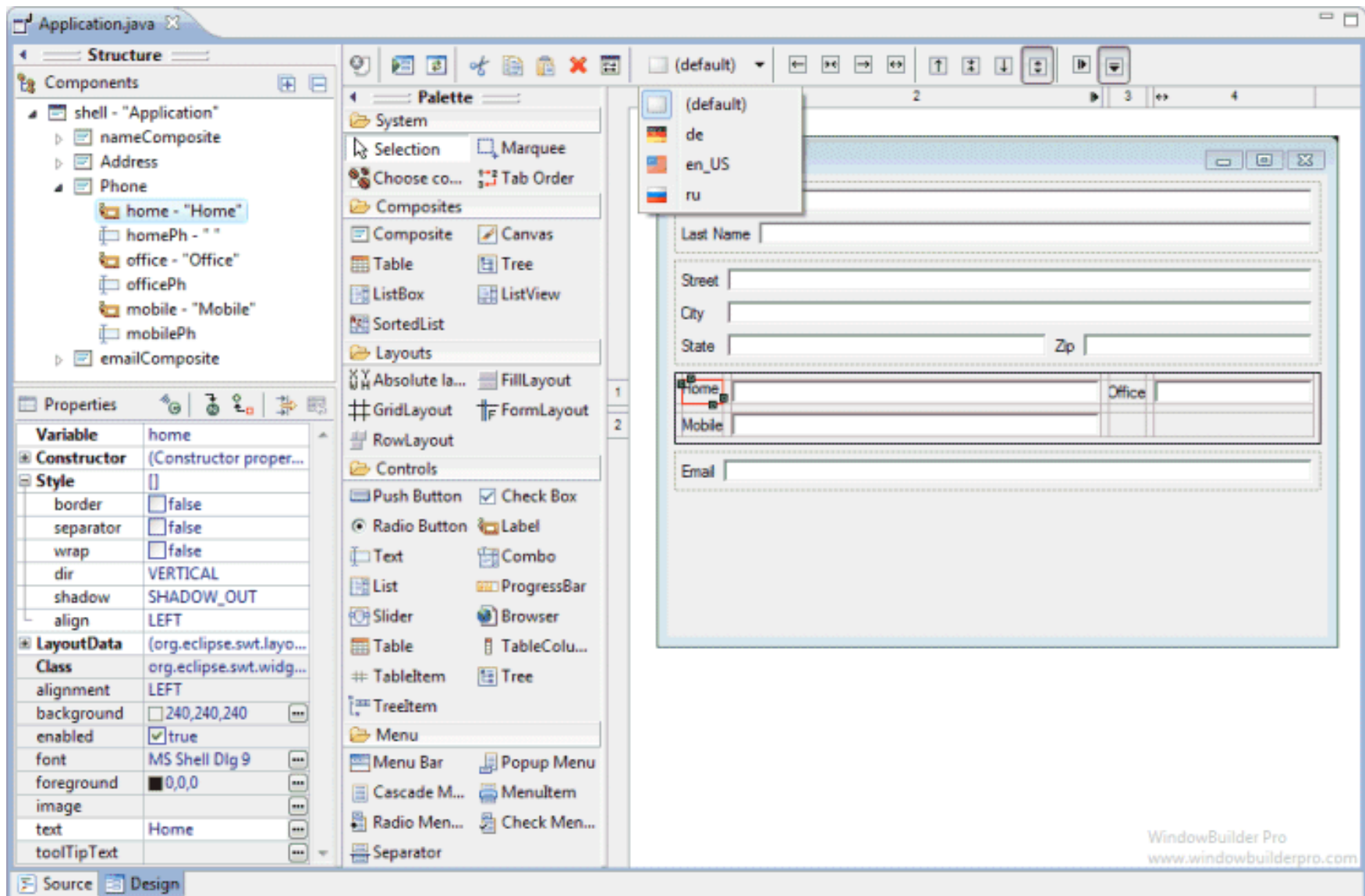
Opção 2

Java Development Kit (JDK)

Integrated Development Environment (IDE) Eclipse

Windows Builder

Interface Gráfica



Interface Gráfica - SWING

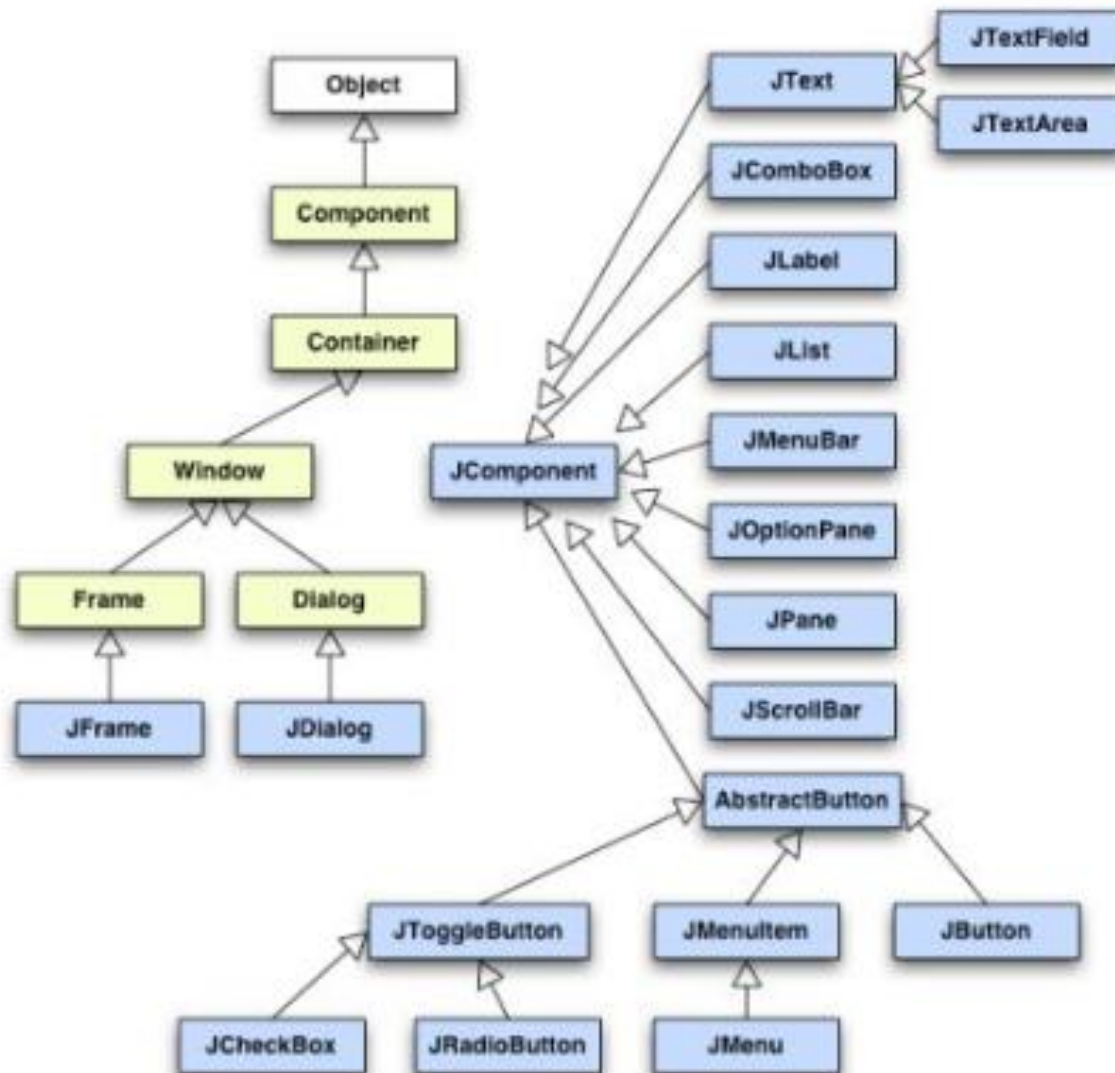
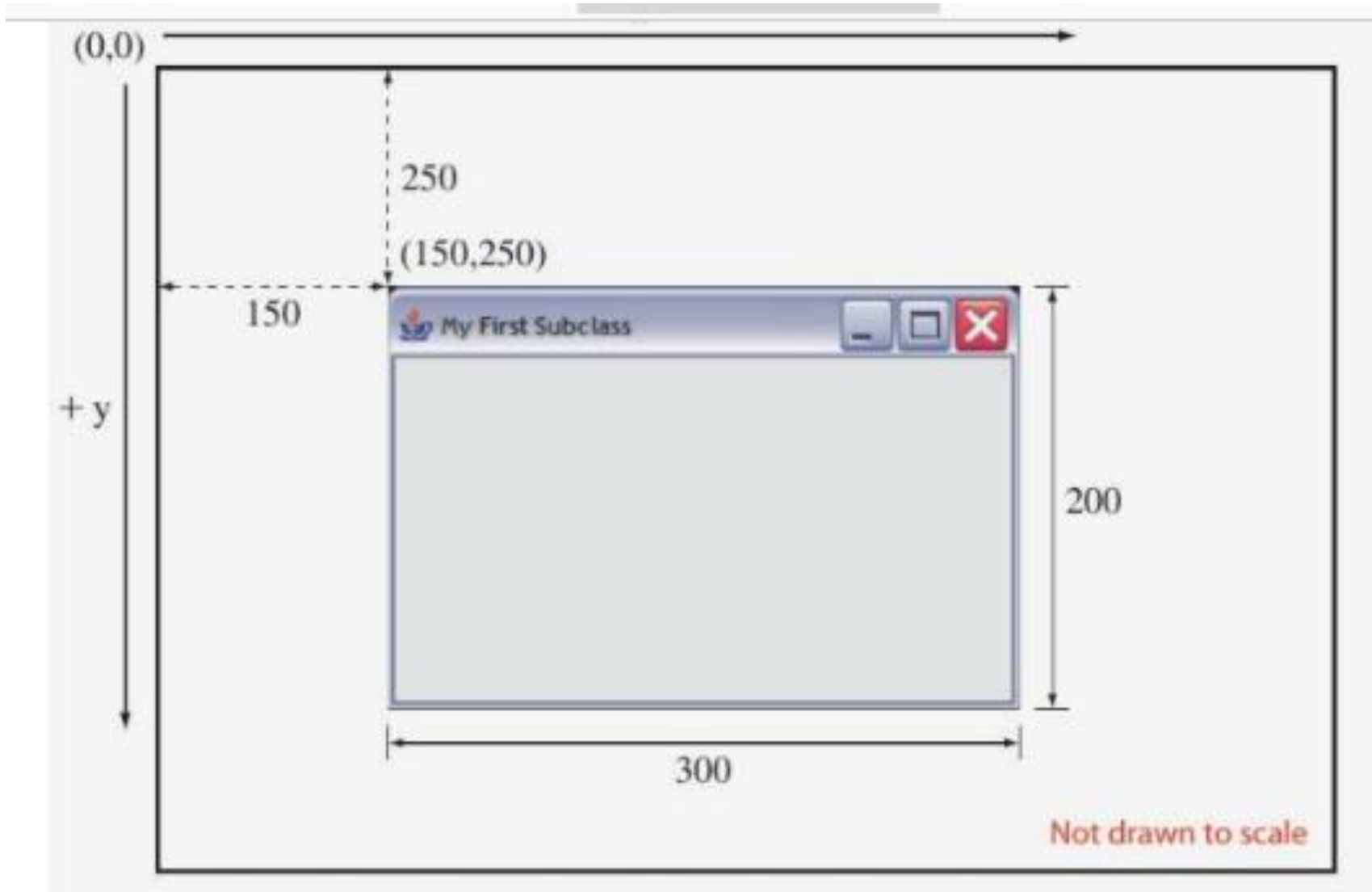
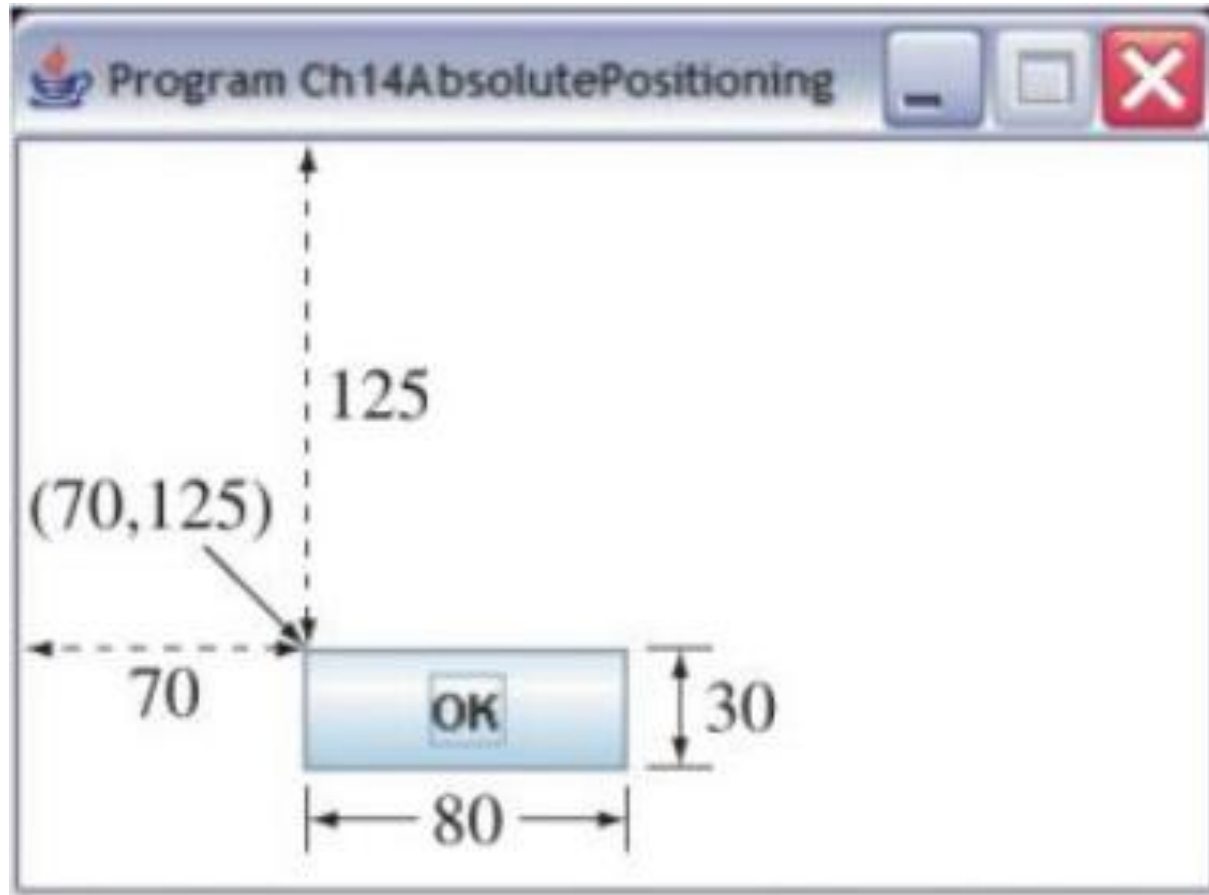


Figura 1: Diagrama resumido das classes AWT (amarelo) e Swing (azul).

Interface Gráfica - SWING



Interface Gráfica - SWING



Interface Gráfica - SWING

The image shows a Java Swing window titled "Sistema de cadastro" with a standard Mac OS X-style title bar (blue with red, yellow, and green buttons). The window contains a registration form with the following components and labels:

- Nome:** A text input field labeled with `JTextField` on the right and `JLabel` on the left.
- Endereço:** A text input field labeled with `JTextField` on the right and `JLabel` on the left.
- Telefone:** A formatted text input field (containing "(") labeled with `JFormattedTextField` on the right and `JLabel` on the left.
- CPF:** A formatted text input field (containing "...-") labeled with `JFormattedTextField` on the right and `JLabel` on the left.
- Tipo de sangue:** A dropdown menu showing "A" labeled with `JComboBox` on the right and `JLabel` on the left.
- Fator RH:** A dropdown menu showing "+" labeled with `JComboBox` on the right and `JLabel` on the left.
- Curso:** A dropdown menu showing "Ciência da Computação" labeled with `JTextField` on the right and `JLabel` on the left.
- Contato de emergência:** A text input field labeled with `JTextField` on the right and `JLabel` on the left.
- Telefone:** A second formatted text input field (containing "(") located below the emergency contact field.
- Buttons:** Two buttons at the bottom labeled "Inserir" and "Cancelar", both labeled with `JButton` above them.
- Window Frame:** The entire window is labeled `JFrame` at the bottom right.

Interface Gráfica

Mas mesmo com essas ferramentas é **necessário** entender **como os componentes são alocados na tela** e como são feitos os **processos de posicionamento, controle de eventos**



Em caso de **manutenção** ou outros problemas **você será capaz de fazer as correções sem depender de uma ferramenta específica.**

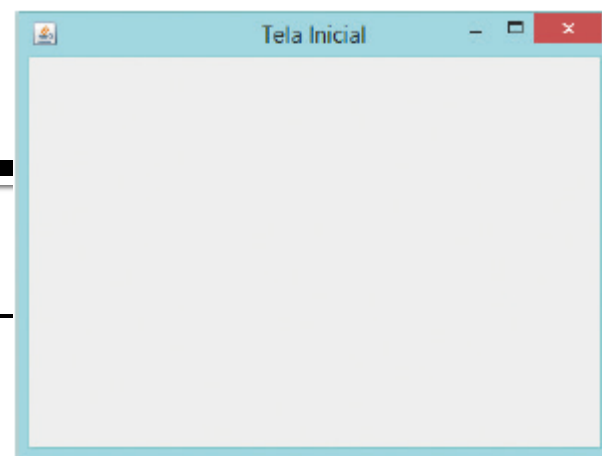


Muitos componentes precisam ser criados em tempo de execução, e isso só é possível com a programação.

Interface Gráfica

1. Criar uma relação de **especialização** com a classe que **representa sua tela**.
2. No construtor, instanciar, **configurar e posicionar** os itens na tela.
3. Declarar como **atributos** os elementos que serão adicionados à tela.
4. Definir a forma de **alocação** dos elementos gráficos na tela.
5. Tratar os eventos dos componentes para tratar as ações do usuário com a interface gráfica.

Interface Gráfica



```
1. package view;
2. import javax.swing.JFrame;
3.
4. public class InterfaceGrafica extends JFrame{
5.     public InterfaceGrafica() {
6.         setSize(400,500);
7.         setTitle("Tela Inicial");
8.         setVisible(true);
9.     }
10.    public static void main(String[] args) {
11.        InterfaceGrafica telaInicial = new InterfaceGrafica();
12.    }
```

Antes de iniciar...

É necessário instalar:

Java Development Kit (JDK)

<https://www.oracle.com/java/technologies/downloads/#jdk18-windows>

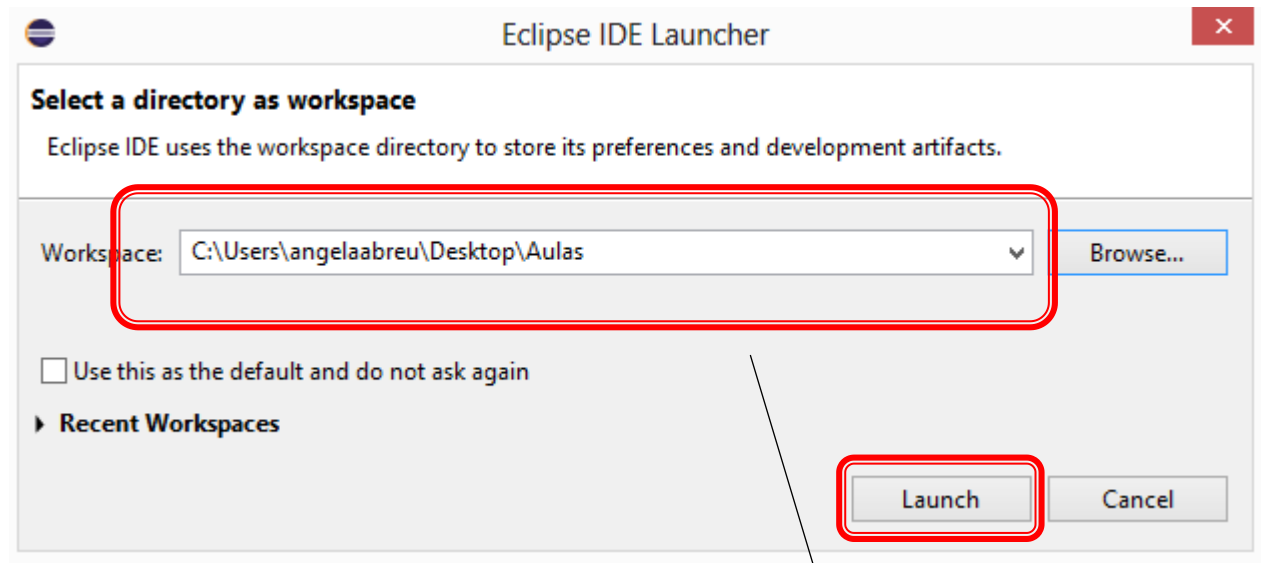
Integrated Development Environment (IDE) Eclipse

<https://www.eclipse.org/downloads/download.php?file=/oomph/epp/2022-03/R/eclipse-inst-jre-win64.exe>

Interface Gráfica



Interface Gráfica



Diretório que irá armazenar o seu projeto.



Welcome to the Eclipse IDE for Java Developers

**Review IDE configuration settings**

Review the IDE's most fiercely contested preferences

**Create a Hello World application**

A guided walkthrough to create the famous Hello World in Eclipse

**Create a new Java project**

Create a new Java Eclipse project

**Checkout projects from Git**

Checkout Eclipse projects hosted in a Git repository

**Import existing projects**

Import existing Eclipse projects from the filesystem or archive

**Launch the Eclipse Marketplace**

Enhance your IDE with additional plugins and install your Marketplace favorites

**Overview**

Get an overview of the features

**Tutorials**

Go through tutorials

**Samples**

Try out the samples

**What's New**

Find out what is new

File Edit Source Refactor Navigate Search Project Run Window Help

New **Alt+Shift+N** ▸

Open File...

Open Projects from File System...

Recent Files ▸

Close Editor **Ctrl+W**

Close All Editors **Ctrl+Shift+W**

Save **Ctrl+S**

Save As...

Save All **Ctrl+Shift+S**

Revert

Move...

Rename... **F2**

Refresh **F5**

Convert Line Delimiters To ▸

Print... **Ctrl+P**

Import...

Export...

Properties **Alt+Enter**

Switch Workspace ▸

Restart

Exit

Java Project

Create a Java project

Package

Class

Interface

Enum

Record

Annotation

Source Folder

Java Working Set

Folder

File

Untitled Text File

JUnit Test Case

Other... **Ctrl+N**

Problems × @ Javadoc Declaration

0 items

Description	Resource	Path	Location	Type

Create a Java Project

Create a Java project in the workspace or in an external location.

Project name:

☒ Use default location

Location:

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE 'jdk-18' and workspace compiler preferences

[Configure...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files

[Configure...](#)

Working sets

☐ Add project to working sets

Working sets:

Module

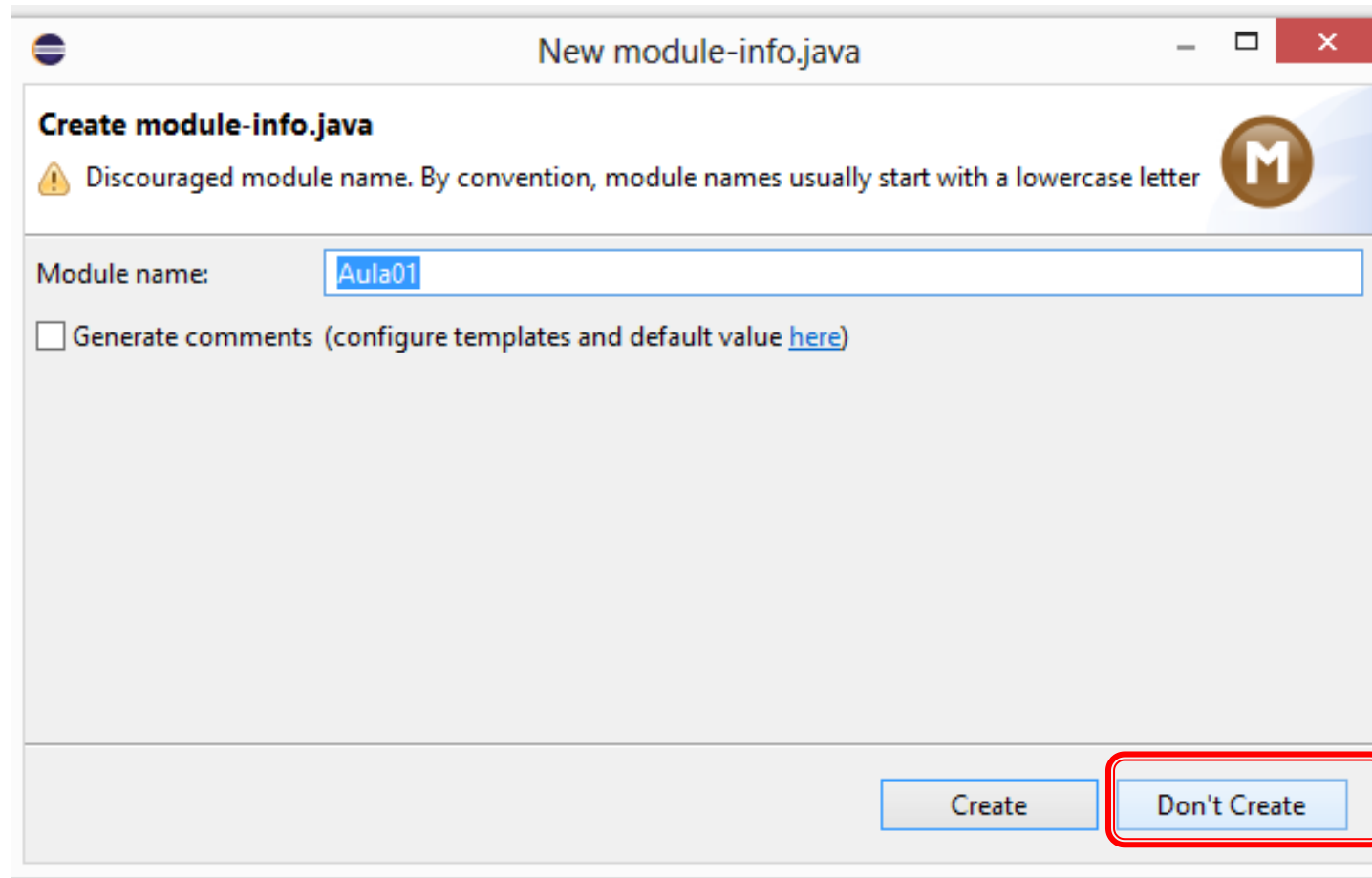
☒ Create module-info.java file



< Back

Next >

Interface Gráfica



The image shows a dialog box titled "New module-info.java". At the top, there is a warning icon and text: "Discouraged module name. By convention, module names usually start with a lowercase letter". To the right of this text is a circular icon with the letter "M". Below the warning, there is a text field labeled "Module name:" containing the text "Aula01". Underneath the text field is a checkbox labeled "Generate comments (configure templates and default value [here](#))". At the bottom right of the dialog, there are two buttons: "Create" and "Don't Create". The "Don't Create" button is highlighted with a red rectangular border.

New module-info.java

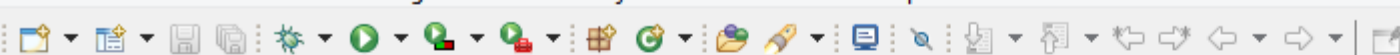
Create module-info.java

⚠ Discouraged module name. By convention, module names usually start with a lowercase letter

Module name:

☐ Generate comments (configure templates and default value [here](#))

Create Don't Create



Package Explorer

Package Explorer

Aut...

New

Go Into

Open in New Window

Open Type Hierarchy F4

Show In Alt+Shift+W

Copy Ctrl+C

Copy Qualified Name

Paste Ctrl+V

Delete Delete

Build Path

Source Alt+Shift+S

Refactor Alt+Shift+T

Import...

Export...

Refresh F5

Close Project

Assign Working Sets...

Coverage As

Run As

Debug As

Restore from Local History...

Team

Compare With

Configure

Java Project

Project...

Package

Class Create a Java class

Interface

Enum

Record

Annotation

Source Folder

Java Working Set

Folder

File

Untitled Text File

JUnit Test Case

Other... Ctrl+N

Problems x @ Javadoc Declaration

Errors, 1 warning, 0 others

Description	Resource	Path	Location	T
Warnings (1 item)				

Interface Gráfica

New Java Class

Java Class
Create a new Java class.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers:
☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass:

Interfaces:

Which method stubs would you like to create?
☒ `public static void main(String[] args)`
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Interface Gráfica

```
package view;

import javax.swing.JFrame;

public class InterfaceGrafica extends JFrame {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}
```

Interface Gráfica

```
package view;

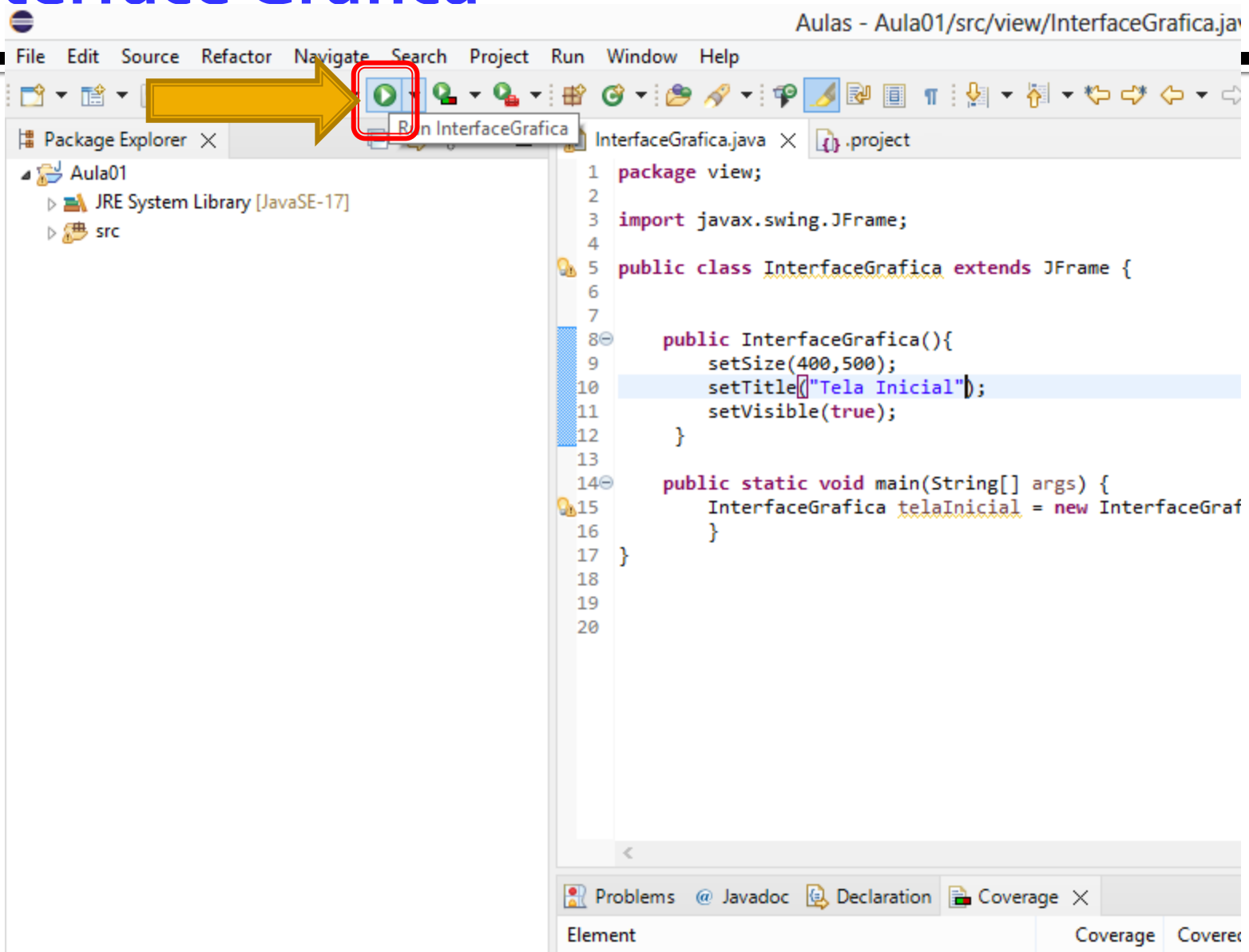
import javax.swing.JFrame;

public class InterfaceGrafica extends JFrame {

    public InterfaceGrafica() {
        setSize(400,500);
        setTitle("Tela Inicial");
        setVisible(true);
    }
    public static void main(String[] args) {
        InterfaceGrafica telaInicial = new InterfaceGrafica();
    }
}
```

,

Interface Gráfica



Tela Inicial

Aulas - Aula01/src/view/InterfaceGrafica.java - Eclipse IDE

File Edit Window Help

InterfaceGrafica.java X .project

package view;

import javax.swing.JFrame;

public class InterfaceGrafica extends JFrame {

public InterfaceGrafica(){

setSize(400,500);

setTitle("Tela Inicial");

setVisible(true);

}

public static void main(String[] args) {

InterfaceGrafica telaInicial = new InterfaceGrafica();

}

Problems Javadoc Declaration Coverage X

Element	Coverage	Covered Instructio...	Missed Instruct

Interface Gráfica

```
1. package view;
2. import javax.swing.JFrame;
3. import javax.swing.JLabel;
4. import javax.swing.JTextField;

5. public class PrimeiraTela extends JFrame {
6.     private JLabel lblNome;
7.     private JTextField txtNome;

8.     public PrimeiraTela() {
9.         lblNome = new JLabel("Nome");
10.        txtNome = new JTextField();

11.        setSize(400,200);
12.        setTitle("Tela Inicial");
13.        setVisible(true);
14.        setLayout(null);

15.        lblNome.setBounds(10,10,100,25);
16.        txtNome.setBounds(50,10,200,25);
17.        getContentPane().add(lblNome);
18.        getContentPane().add(txtNome);
19.    }

20.    public static void main(String[] args) {
21.        PrimeiraTela t1 = new PrimeiraTela();
22.    }
--
```



Interface

New Java Class

Java Class
Create a new Java class.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass:

Interfaces:

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

```
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;

public class PrimeiraTela extends JFrame {

    //Atributos
    private JLabel lbNome;
    private JTextField txtNome;

    public PrimeiraTela(){

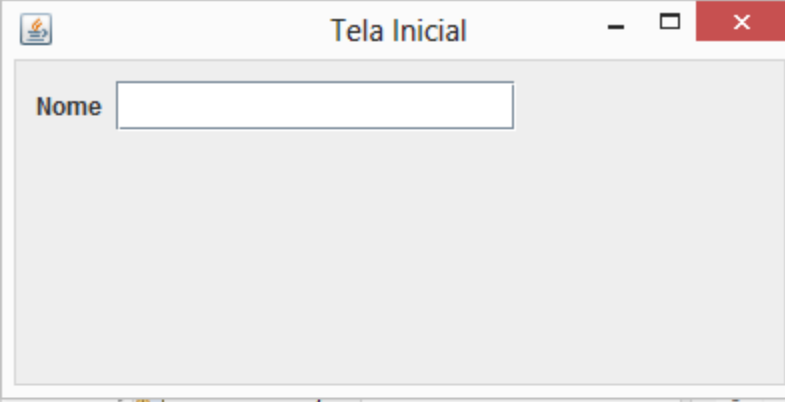
        //Criar o Label e a caixa de texto
        lbNome = new JLabel("Nome");
        txtNome = new JTextField();

        setSize(400,200);
        setTitle("Tela Inicial");
        setVisible(true); //deixar a janela visível
        setLayout(null); //limpar o layout para configurar os componentes nos locais corretos

        //Configurar a área ("retângulo") do Label e da caixa de texto
        lbNome.setBounds(10,10,100,25);
        txtNome.setBounds(50, 10, 200, 25);
        //Adicionar o Label e a caixa de texto à tela
        getContentPane().add(lbNome);
        getContentPane().add(txtNome);

    }

    public static void main(String[] args) {
        PrimeiraTela t1 = new PrimeiraTela();
        t1.setVisible(true);
    }
}
```



Aulas - Aula01/src/view/PrimeiraTela.java - Eclipse IDE

```
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;

public class PrimeiraTela extends JFrame {

    //Atributos
    private JLabel lbNome;
    private JTextField txtNome;

    public PrimeiraTela(){

        //Criar o Label e a caixa de texto
        lbNome = new JLabel("Nome");
        txtNome = new JTextField();

        setSize(400,200);
        setTitle("Tela Inicial");
        setVisible(true); //deixar a janela visível
        setLayout(null); //limpar o layout para configurar os componentes nos locais

        //Configurar a área ("retângulo") do Label e da caixa de texto
        lbNome.setBounds(10,10,100,25);
        txtNome.setBounds(50, 10, 200, 25);
        //Adicionar o Label e a caixa de texto à tela
        getContentPane().add(lbNome);
        getContentPane().add(txtNome);

    }

    public static void main(String[] args) {
        PrimeiraTela t1 = new PrimeiraTela();
    }
}
```

Problems | Javadoc | Declaration | Coverage

0 errors, 5 warnings, 1 other

Description	Resource	Path	Location	Type
-------------	----------	------	----------	------

Interface Gráfica

The image shows a Java Swing window titled "Tela Inicial". It contains three input fields and a button. The first field is labeled "Nome" and is a standard text field. The second field is labeled "CPF" and is a formatted text field with a mask. The third field is labeled "Tipo de usuário" and is a dropdown menu currently showing "Administrador". Below these fields is a large blue button labeled "Enviar".

Componente	Classe Java
Nome	TextField
CPF	FormattedTextField
Tipo de usuário	ComboBox
Enviar	Button

TextField

FormattedTextField

ComboBox

Button

```
import java.awt.Container; //biblioteca para
containers

import java.text.ParseException;

import javax.swing.*; //simplificando a
inclusão de bibliotecas

import javax.swing.text.MaskFormatter;

public class PrimeiraTela extends JFrame {

    private JLabel lblNome;
    private JTextField txtNome;
    private JLabel lblCPF;
    private JFormattedTextField txtCPF;
    private JLabel lblTipo;
    private JComboBox cmbTipo;
    private final String[] tiposUsuarios =
    {"Adminstrador", "Geral"};
    private JButton btnOK;
    private Container ctn;

    public PrimeiraTela() {
        setSize(400, 300);
        setTitle("Tela Inicial");
        ctn = getContentPane();
        lblNome = new JLabel("Nome");
        txtNome = new JTextField();
        lblCPF = new JLabel("CPF");
        try {
            txtCPF = new JFormattedTextField(new
            MaskFormatter("###.###.###-##"));
        }
    }
}
```

```
    } catch (ParseException e) {  
        e.printStackTrace();  
    }  
    lblTipo = new JLabel("Tipo de usuário");  
    cmbTipo = new JComboBox(tiposUsuarios);  
    btnOK = new JButton("Enviar");  
    ctn.setLayout(null);  
    lblNome.setBounds(0, 0, 100, 25);  
    txtNome.setBounds(150,0,200,25);  
    lblCPF.setBounds(0,50,100,25);  
    txtCPF.setBounds(150,50,200,25);  
    lblTipo.setBounds(0,100,200,25);  
    cmbTipo.setBounds(150,100,200,25);  
    btnOK.setBounds(150, 150, 100, 100);  
    ctn.add(lblNome);  
    ctn.add(txtNome);  
    ctn.add(lblCPF);  
    ctn.add(txtCPF);  
    ctn.add(lblTipo);  
    ctn.add(cmbTipo);  
    ctn.add(btnOK);  
    setVisible(true);  
    setDefaultCloseOperation(JFrame.EXIT_  
ON_CLOSE);  
}
```

```
public static void main(String[] args) {  
    PrimeiraTela t1 = new PrimeiraTela();  
}
```

Tela Inicial

Nome

CPF

Tipo de usuário Administrador ▼

Aulas - Aula01/src/view/PrimeiraTela2.java - Eclipse IDE

```
ctn = getContentPane();
lblNome = new JLabel("Nome");
txtNome = new JTextField();
lblCPF = new JLabel("CPF");
try {
    txtCPF = new JFormattedTextField(new MaskFormatter("###.###"))
}
catch (ParseException e) {
    e.printStackTrace();
}

lblTipo = new JLabel("Tipo de usuário");
cmbTipo = new JComboBox(tiposUsuarios);
btnOK = new JButton("Enviar");
ctn.setLayout(null);
lblNome.setBounds(0, 0, 100, 25);
txtNome.setBounds(150, 0, 200, 25);
lblCPF.setBounds(0, 50, 100, 25);
txtCPF.setBounds(150, 50, 200, 25);
lblTipo.setBounds(0, 100, 200, 25);
cmbTipo.setBounds(150, 100, 200, 25);
btnOK.setBounds(150, 150, 100, 100);
ctn.add(lblNome);
ctn.add(txtNome);
ctn.add(lblCPF);
ctn.add(txtCPF);
ctn.add(lblTipo);
ctn.add(cmbTipo);
ctn.add(btnOK);
setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public static void main(String[] args) {
    PrimeiraTela2 t1 = new PrimeiraTela2();
}

}
```

Problems @ Javadoc Declaration Coverage

Interface Gráfica

Máscaras para o JFormattedTextField:

- # para representar a entrada de números.
- U para letras em caixa alta.
- L para letras em caixa baixa.
- A para qualquer número ou letra.
- ? para qualquer caractere.
- * para qualquer elemento.
- H para entrada em hexadecimal.

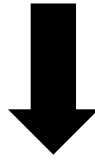
Cuidado...

Os códigos vão ficando cada vez maiores!!!

- Organização;
- Entender cada linha do programa!
- Entender a sequencia lógica.

Interface Gráfica

Sobre os *package* é *sempre* interessante manter as classes de interface no mesmo pacote



Manter a **alta coesão** e **baixo acoplamento**

Coesão → classes, pacotes e métodos têm funções delimitadas, relacionando apenas a um propósito

Alto acoplamento → funcionamento independente de outras classes para executar sua função

Exercício 1

The image shows a Java Swing window titled "Tela Inicial". Inside the window, there are three input fields and one button. The first field is labeled "Nome" and is a standard text field. The second field is labeled "CPF" and is a formatted text field with a mask. The third field is labeled "Tipo de usuário" and is a dropdown menu currently showing "Administrador". Below these fields is a large blue button labeled "Enviar".

Componente	Classe Java
Nome	TextField
CPF	FormattedTextField
Tipo de usuário	ComboBox
Enviar	Button

TextField

FormattedTextField

ComboBox

Button

Exercício 2

A empresa pela qual você foi contratado recebeu um pedido de uma universidade para o **desenvolvimento de uma interface gráfica** para um sistema de matrícula.

Você está na equipe de desenvolvimento do *front-end*, e foi *destacado para* atender essa demanda. Esse sistema de matrícula já possui cerca de 30 anos e ainda não passou pelo processo de modernização.

O líder da sua equipe pediu para que você faça o **código que apresente os componentes visuais principais da tela de cadastro de aluno**, e essa tela deve conter os seguintes dados do universitário:

Exercício 2

- Nome (*label* e *campo de texto*).
- Endereço (*label* e *campo de texto*).
- Código de área e telefone (*label* e *campo de texto* com máscara de formatação).
- Tipo sanguíneo e fator Rh (*label* e *caixa de seleção*).
- Curso atual (*label* e *caixa de seleção*).
- Nome e telefone do contato de emergência (*labels*, *caixa de texto* e caixa de texto com máscara de formatação).

label

caixa de texto

caixa de seleção

Exercício 2

The image shows a Java Swing window titled "Sistema de cadastro" with a standard Mac OS-style title bar (blue with red, yellow, and green buttons). The window contains a registration form with the following components:

- Nome:** A text field preceded by a **JLabel**.
- Endereço:** A text field preceded by a **JLabel**.
- Telefone:** A formatted text field (JFormattedTextField) showing "()" preceded by a **JLabel**.
- CPF:** A formatted text field (JFormattedTextField) showing "...-" preceded by a **JLabel**.
- Tipo de sangue:** A dropdown menu (JComboBox) showing "A" preceded by a **JLabel**.
- Fator RH:** A dropdown menu (JComboBox) showing "+" preceded by a **JLabel**.
- Curso:** A dropdown menu (JComboBox) showing "Ciência da Computação" preceded by a **JLabel**.
- Contato de emergência:** A text field preceded by a **JLabel**.
- Telefone:** A second formatted text field (JFormattedTextField) showing "()" preceded by a **JLabel**.
- Buttons:** Two buttons at the bottom, "Inserir" and "Cancelar", both preceded by **JButton** labels.

The entire window is a **JFrame**.

Exercício 3

Sistemas do tipo *Enterprise Resource Planning (ERP)* envolvem a construção e integração de vários subsistemas e são largamente utilizados por empresas dos mais diversos ramos. Você, como programador, ficou responsável **por criar os campos do formulário que recebe dados de fornecedores.**

Para isso, além de outros diversos campos, é necessário fazer um campo para cadastro de CEP, CNPJ e código de fornecedor. Os campos de CEP e CNPJ são numéricos, por exemplo:

- CEP: 12912-010
- CNPJ: 50.364.671/0001-00

Já o campo de código de fornecedor apresenta uma formação em que os **dois primeiros elementos são letras de caixa alta, um separador representado por um traço, os próximos cinco elementos são números e o último elemento é uma letra em caixa baixa**: **AA- 12345a**.

Para essa tarefa apenas mostre como seria a criação da instância desses três campos.

Exercício 3

Para cada campo é necessário utilizar uma instância da classe `JFormattedTextField` com um elemento de `MaskFormatter` diferente:

- CEP: `JFormattedTextFieltxtCEP = new JFormattedTextField(new MaskFormatter("#####-##"));`
- CNPJ: `JFormattedTextFieltxtCNPJ = new JFormattedTextField(new MaskFormatter("##.###.####/####-##"));`
- Código de fornecedor: `JFormattedTextFieltxtCodFor necedor = new JFormattedTextField(new MaskFormatter("UU-#####L"));`

Na máscara do *JFormattedTextField* o **U** representa a entrada de **letras que são tratadas para caixa alta**, o **L**, a entrada de **letras com caixa baixa**, e o **#** **representa a entrada de um número**



Muito Obrigada!

Prof^a. Angela Abreu Rosa de Sá, Dr^a.

Contato: angelaabreu@gmail.com